

Szegedi Tudományegyetem
Informatikai Intézet

SZAKDOLGOZAT

Pető József

2025



Szegedi Tudományegyetem
Természettudományi és Informatikai Kar

A numerikus Radon-transzformáció vizsgálata

Szakdolgozat

Készítette:
Pető József
programtervező informatikus
BSc szakos hallgató

Témavezető:
Domokos Zoltán
tudományos segédmunkatárs

Szeged
2025

Feladatkiírás

A hallgató feladata a numerikus Radon-transzformáció vizsgálata. Praktikus okoknál fogva az említett művelet implementációja során legtöbbször azt feltételezzük, hogy a transzformálandó dobozfüggvények függvények lineáris kombinációjaként előállítható.

Ugyanakkor egy aktívan kutatott témában szükséges lenne a módszer általánosítására több, a lépcsős függvényektől eltérő bázis esetére. A szakdolgozat célja legalább egy, de lehetőleg több alternatív bázisra elvégezni ezt, majd Python nyelven írt numerikus szimulációkkal demonstrálni a kidolgozott módszer helyességét. Opcionálisan további feladat lehet még az implementált algoritmusok párhuzamosítása a Numba könyvtár segítségével.

Tartalmi összefoglaló

A szakdolgozat célja, hogy egy átfogó és alapos elemzést nyújtson a Radon-transzformáció különböző implementációs lehetőségeiről és különbségeiről, valamint azok hatékonyságáról. A modern képalkotó technológiák egyik alapvető eszköze a számítógépes tomográfia, amely a Radon-transzformáción alapul. Az említett transzformáció olyan lineáris leképezés, amely egy függvény vonalintegráljait számítja ki különböző irányú egyenesek mentén. A Radon-transzformáció kulcsszerepet játszik nemcsak az orvosi képalkotásban, hanem az ipari anyagvizsgálatban és a geofizikai kutatásokban is, így széles körben alkalmazzák.

A gyakorlatban az analitikus Radon-transzformáció diszkrét közelítésével kell dolgoznunk. Ennek kétféle változatát hasonlítjuk össze a szakdolgozatban. Az egyik esetben azt feltételezzük, hogy a Radon-transzformációt dobozfüggvények lineáris kombinációján alkalmazzuk, a másik esetben pedig körfüggvények alkotta bázist feltételezünk.

Az utóbbi esetben a körök középpontja, sugara és intenzitása alapján határozzuk meg a vonalintegrálokat. Ez pontosabb rekonstrukciót tesz lehetővé akkor, ha az eredeti függvény jobban közelíthető körfüggvények lineáris kombinációjaként. Az említett megoldás részletes összehasonlításon megy keresztül a hagyományos, az eredeti függvényt dobozfüggvények lineáris kombinációjának feltételező módszerrel. Az eljárások és azok tesztelési eredményeinek elemzése rávilágít arra, hogy a körfüggvény alapú megközelítés milyen mértékben tér el a másik módszertől, illetve hogy a rekonstrukció minősége mennyire különbözik. Az összehasonlítások során a TomoPy könyvtárat is felhasználtam, ami hozzájárult az eredmények ellenőrzéséhez. Ezáltal átfogó képet kapunk arról, mennyire alkalmazható ez a megközelítés a valós képalkotási problémák megoldásában.

A tesztképeket egy szkript állítja elő, amiben a megjelenő körök minden paramétere lebegőpontos értéként definiált, így biztosítva, hogy minél változatosabb tesztmintákat kapjunk. A képeken alkalmazott Radon-transzformáció eredményeinek különbségei vizuálisan és számszerűen is kiértékelésre kerülnek, amiket olyan

mérőszámok jellemeznék, mint a Mean Squared Error (MSE), Mean Absolute Error (MAE), valamint a Maximum Absolute Error (MaxAE). Az eredmények ezen felül különbségképek és hisztogramok formájában is megtekinthetők. Ezt követően a szinogram alapján, a TomoPy csomagban implementált gridrec rekonstrukciós algoritmus (cosine szűrővel) alkalmazásával visszanyerhető az eredeti kép egy közelítése. A rekonstruált képek és az eredeti referenciaképek közti különbségek elemzésére is sor került, amely során szemléltetésre kerülnek a különböző megoldások közötti eltérések és hiba értékek.

Mivel a CPU alapú feldolgozás jelentős számításigénnyel rendelkezik, ezt elsősorban a Radon-transzformáció működésének vizualizálására és tesztelésére alkalmas. A GPU alapú megoldás, a Numba CUDA alkalmazásával, jelentősen kihasználja a párhuzamos számítás előnyeit, ami nagymértékben lerövidíti a futásidőt.

Tartalomjegyzék

Feladatkiírás	3
Tartalmi összefoglaló	4
Elméleti háttér	8
1.1 Folytonos Radon-transzformáció	8
1.2 Diszkrét Radon-transzformáció	8
1.3 Dobozfüggvény vonalintegráljának számítása.....	11
1.4 Diszkrét Radon-transzformáció körfüggvényekkel	14
1.5 Körfüggvény vonalintegráljának számítása	16
Alkalmazott eszközök	21
2.1 Python és Python csomagok.....	21
2.2 Numba CUDA – párhuzamos számítás.....	22
A program felépítése	23
3.1 Mappaszerkezet.....	23
3.2 Bázisfüggvény választásától független eljárások.....	25
3.3 Dobozfüggvény tartója és a vetítésűgár közti metszések számítása.....	26
3.4 Körfüggvény tartója és a vetítésűgár közti metszések számítása	27
3.5 Egyéb működésbeli eltérések	29
3.6 GPU megvalósítás.....	29
Mérési eredmények	31
4.1 Néhány gondolat az eredmények kapcsán	31
4.2 Szinogram vizsgálata	32
4.3 Kép rekonstruálása a szinogramból	38
4.4 A vetületi szögek számának jelentősége rekonstrukció esetén	41
4.5 Futásidő elemzés	44
Összefoglalás	47
Irodalomjegyzék	48
Nyilatkozat	50
Köszönetnyilvánítás	51
Mellékletek	52

1. fejezet

Elméleti háttér

1.1 Folytonos Radon-transzformáció

A Radon-transzformáció egy olyan lineáris leképezés, amely egy adott függvényhez (például egy kép intenzitásfüggvényéhez) a függvény vonalintegráljainak eredményét rendeli egy egyenes sereg mentén. Ez különösen fontos az orvosi és ipari képalkotásban (pl. CT).

A folytonos Radon-transzformáció egy kétdimenziós $f(x, y)$ függvény esetén [2],[13]:

$$(Rf)(\varphi, t) = \int_{-\infty}^{+\infty} f(t \cos(\varphi) - q \sin(\varphi), t \sin(\varphi) + q \cos(\varphi)) dq \quad (1.1)$$

A képlet tartalmazza az egyenes irányszögét φ , az origótól mért távolságát t , és az egyenes bejárásának q paraméterét. Vagyis a Radon-transzformáció, a φ és t paraméterekkel jellemezhető egyenesek mentén a függvény vonal menti integrálja. Fontos megjegyezni, hogy mivel egy adott irányban vett vonalintegrál független a 180° -os elforgatástól (radiánban π), a szögértékek $[0, \pi)$ -en értelmezettek.

1.2 Diszkrét Radon-transzformáció

A rekonstrukciós algoritmusok teszteléséhez gyakran szokás szimulált bemeneti adatokat használni. Ilyenkor, a szinogram előállításánál, a folytonos Radon-transzformáció diszkrét közelítésére van szükség. Általában az eredeti függvényt dobozfüggvények lineáris kombinációjával közelítjük. Ekkor, ahogy lentebb látni fogjuk, az értelmezési tartományban egy adott egyenes és a pixelek metszéspontjai közötti távolságok súlyozott összegeként előállnak a vonalintegrálok eredményeit.

A képletek egyszerűbb megjelenése, illetve az átláthatóság kedvéért nem külön a

kép szélessége és magassága szerint járjuk be a képet, hanem azt egy egydimenziós tömbként kezeljük. Ezáltal a k index kerül bevezetésre ami a pixelek sorszámát jelenti.

A diszkrét eset képletbe foglalásához a folytonos alak egyszerűsített változatát kell venni. Mivel a Radon-transzformáció lineáris, a homogenitás miatt a c_k konstansok kiemelhetők, a disztributivitás pedig megengedi, hogy az egyes komponensekre külön-külön elvégezhető legyen a transzformáció. Így adódik:

$$R(f)(x, y) = c_0 R(f_0)(x, y) + \dots + c_k R(f_k)(x, y) \quad (1.2)$$

Ez azt jelenti, hogy a diszkrét képen végzett művelet során először minden pixelre külön alkalmazzuk a transzformációt, majd az eredményeket a megfelelő konstansszorzókkal súlyozzuk és összegezzük.

A folytonos jel diszkrét reprezentációjához bevezetjük a dobozfüggvényeket, jelölésben $rect_k$. Itt a k index azonosítja a dobozfüggvény középpontjának eltolását, úgy ahogy azt az (1.3)-as képletben látjuk. Különböző mértékben eltolt kétdimenziós dobozfüggvények lineáris kombinációja egy pixelekből álló szürkeárnyaltos képnek feleltethető meg.

A kétdimenziós dobozfüggvény [15] két egydimenziós dobozfüggvény szorzataként írható fel. Egy adott k indexű pixel középpontjába ($P_x(k)$, $P_y(k)$) eltolt egydimenziós dobozfüggvény matematikai leírása:

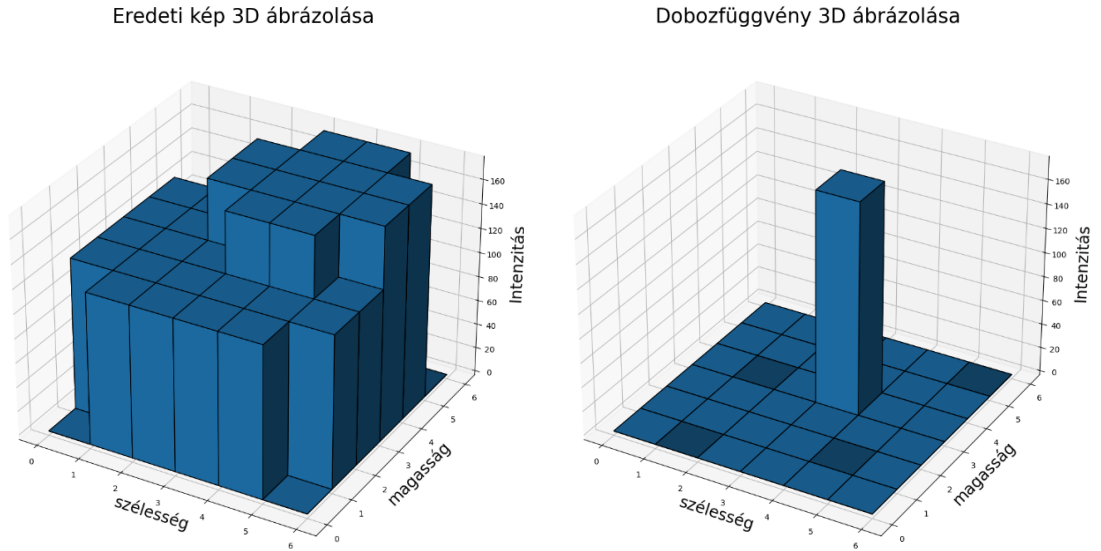
$$f_k(x) = \begin{cases} 1, & \text{ha } |x - P_x(k)| < \frac{\Delta x}{2} \\ 0.5, & \text{ha } |x - P_x(k)| = \frac{\Delta x}{2} \\ 0, & \text{egyébként} \end{cases} \quad (1.3)$$

A másik változó esetében a szerkezet változatlan, csupán a jelölése tér el, ami $g_k(y)$. Ennek megfelelően a kétdimenziós $rect$ függvény úgy áll elő, hogy az adott (x, y) ponthoz az egydimenziós függvények értékeinek szorzatát rendeljük:

$$rect(x, y) = f(x) g(y) \quad (1.4)$$

Érdemes megemlíteni, hogy mivel a kép pixelekből áll, bonyolultabb, például kör alakú régiókat nem lehet pontosan előállítani. Csupán közelíteni tudjuk, hiszen a kör nem rekonstruálható tökéletesen több négyzet összegzésével, így az ilyen formák esetén mindig valamilyen fokú torzulásra kell számítani. Az 1.1. ábra kétdimenziós

dobozfüggvények lineáris kombinációjaként előállítható függvényt láthatunk.



1.1. ábra. A *rect* eltolt változatainak lineáris kombinációja a balra látható, ahol minden pixelhez egy középpontba helyezett dobozfüggvény tartozik, amely a pixel területén a lineáris kombinációhoz tartozó c_k súlyozott értéket, azon kívül pedig nulla értéket vesz fel. Jobbra pedig kiemelve a (3,3) koordinátájú pixelhez a hozzá tartozó c_k együtthatóval súlyozott $rect_k$ -el látható.

A kifejezést visszahelyettesítve az (1.2)-ben szereplő kifejezésbe, úgy járunk el, hogy minden egyes *rect* függvényre külön-külön alkalmazzuk a Radon-transzformáció műveletét. Ennek eredményeként az egész függvény Radon-transzformációja úgy áll elő, hogy az egyes dobozfüggvények transzformációit a hozzájuk tartozó megfelelő intenzitás értékekkel súlyozzuk, majd összegezzük végzünk.

$$R(f) = \sum_k c_k R(rect_k)(x, y) \quad (1.5)$$

Ebből adódik, hogy egy adott (φ, t) paraméterekkel jellemzett egyenesre a Radon-transzformált értékét, az (1.5)-ös és az (1.1)-es kifejezések alapján az alábbi képlettel számíthatjuk:

$$R(f)(\varphi, t) = \sum_k c_k \int_{-\infty}^{\infty} rect_k(t \cos(\varphi) - q \sin(\varphi), t \sin(\varphi) + q \cos(\varphi)) dq \quad (1.6)$$

A képletben az x és y koordinátákat az egyenes mentén paraméterezzük úgy, hogy az egyenes egy pontját $(t \cos(\varphi), t \sin(\varphi))$ adja, míg az egyenest a q paraméter szerint

járjuk be.

Fontos megjegyezni, hogy bár az integrálási határok a képletben $-\infty$ és ∞ között vannak megadva, a $rect_k$ függvény csak a pixelhez tartozó, korlátozott pixeltartományban vesz fel egyet. Ezen kívül a függvény értéke nulla, így a képletben szereplő végtelen integrálási határok mellett kapott eredmény megegyezik az adott tartományba eső vonalszakasz feletti határozott integrállal.

1.3 Dobozfüggvény vonalintegráljának számítása

A diszkrét Radon-transzformáció (1.6) számításához, először a vetítősugar pixelen belüli hosszát számítjuk ki L_i . Első lépésként a sugarat reprezentáló egyenes normálvektorát vesszük, ami a következőképpen fejezhető ki:

$$\mathbf{n} = \begin{bmatrix} \cos(\varphi) \\ \sin(\varphi) \end{bmatrix} \quad (1.7)$$

Eszerint a fenti képletben a φ a vetítés szöge, amely tekinthető egy valós mérés esetén a detektorsor egyenesének az x-tengellyel bezárt szögeként.

Az így meghatározott normálvektor segítségével kiszámítható az adott pixel élének és a vetített sugárnak a metszéspontja, mivel a metszés feltétele vektoregyenletekkel jól leírható. Ennek érdekében a sugarat és a pixelt határoló egyenes a normálvektor segítségével írjuk fel, amely az alábbi formában adható meg:

$$\mathbf{n}_{sugar}^T \mathbf{x} = \mathbf{w}_{sugar} \quad (1.8)$$

$$\mathbf{n}_{pixel}^T \mathbf{x} = \mathbf{w}_{pixel} \quad (1.9)$$

Mivel két egyenes metszéspontját kell meghatároznunk, az (1.8) és (1.9) egyenletek együttesen alkotják a lineáris egyenletrendszert, amelyet megoldva a keresett metszéspont koordinátái adódnak. Ez alapján a metszéspont a következő egyenletből adódik:

$$\mathbf{N} \mathbf{x} = \mathbf{w} \quad (1.10)$$

Ahol az $\mathbf{N}$ mátrix tartalmazza a két egyenes normálvektorának transzponáltját és a $\mathbf{w}$ egy konstans vektor ami az egyenesek eltolásától és a vetítés szögétől függ. Az (1.10)-es

egyenletben az N mátrix és a w vektor külön-külön a következő formában adódnak:

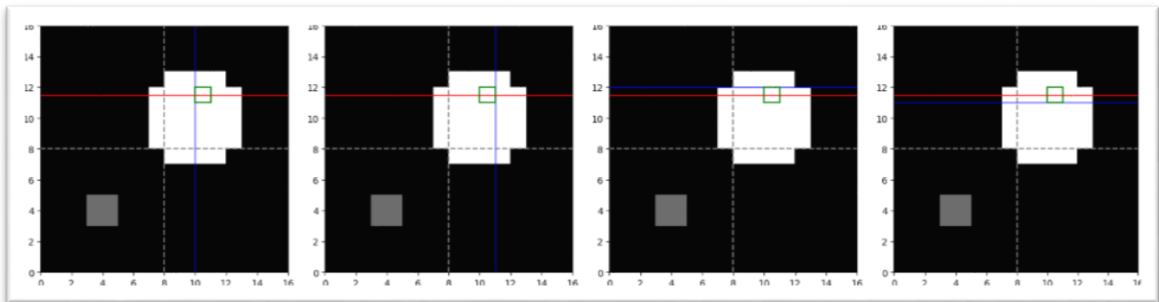
$$N = \begin{bmatrix} \mathbf{n}_{sugár}^T \\ \mathbf{n}_{pixel}^T \end{bmatrix}, \quad w = \begin{bmatrix} \mathbf{n}_{sugár}^T \cdot \mathbf{v}_0 \text{ sugár} \\ \mathbf{n}_{pixel}^T \cdot \mathbf{v}_0 \text{ pixel} \end{bmatrix} \quad (1.11)$$

A w vektor kifejezésében a $\mathbf{v}_0$ egy ismert pont, amely a megfelelő egyenes egyik pontja a térben. Viszont, ha a vetítősugár párhuzamos egy pixel adott élével, akkor az egyenletnek (1.10) nincs megoldása, amennyiben a vetítősugár és a pixelt határoló egyenes nem esik egybe. Ilyen esetben nem kapunk metszéspontot. Mikor a vetítősugár és a pixelt határoló egyenes egybeesik, akkor az (1.10) egyenletrendszer alulhatározott és végtelen sok megoldása van.

A lineáris egyenletrendszert megoldva megkapjuk a két vonal metszéspontját (ha van). Ezután ellenőrizni kell, hogy a metszéspontok az adott pixelt határoló szakaszokra esnek-e, ezzel biztosítva, hogy valóban a pixel élével történő metszést kapjuk. Végül a metszéspontok közötti távolság megadja az L_i -t, vagyis a pixelen belül áthaladó szakasz hosszát.

$$L_i = \sqrt{(p_{1x} - p_{2x})^2 + (p_{1y} - p_{2y})^2} \quad (1.12)$$

Ezt a számítást a vonal mentén minden egyes pixelen elvégezzük, vagyis minden pixel esetében meghatározzuk a vetítősugár által megtett távot. Az így kapott értékeket végül a pixel intenzitásokkal súlyozva összegezzük.

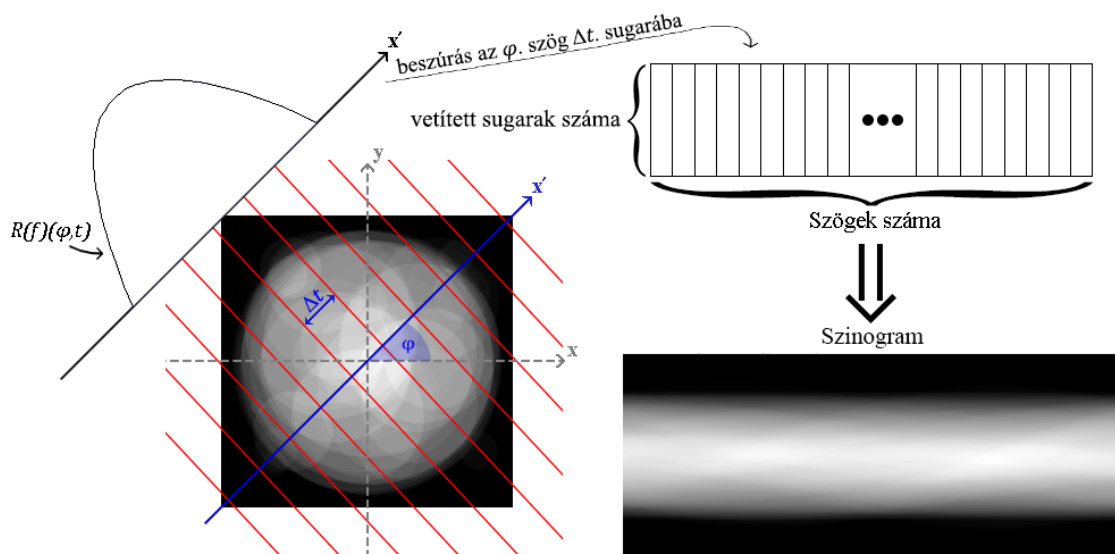


1.2. ábra. A pixel négy oldalának és a vetítősugár metszéspontjának vizsgálatának vizualizációja, ahol a piros vonal a vetítősugár, a kék pedig a pixel meghosszabbított egyenese.

A vetítési sugár paraméterezéséhez φ irányszöveget és t távolságot használjuk. A sugarak mindig úgy vannak elhelyezve, hogy minden pixel fedett legyen bármely irányból, és ne

legyen olyan, hogy két szomszédos pixel határán fut valamely vetítősugár. Ennek megoldására a vetítendő sugár kezdeti értéke fél egység a kép szélétől indulva, mivel egy pixel szélessége egy egységnyi, így a vetítősugár pontosan felezi a pixelt.

Ezt egészen a kép/mátrix méretéig kell iterálni, ezáltal az összes pixelt bejárjuk. Amikor egy szögnél végig léptettük a vetítősugarakat, utána a megváltozott szögű egyenesekre is elvégezzük az ismert eljárást. A detektorsor egyenesének szöge kezdetben az x tengelyre párhuzamos (0°), ami egészen 179° -ig változtatja az értékét, ennek oka, hogy a Radon-transzformáció periodikus tulajdonsága miatt 180° -tól kezdve a számítások redundánsak lennének.



1.3. ábra. A képen a Radon-transzformáció folyamata látható. A balra egy tesztkép helyezkedik el, amelyen φ szögben halad el a detektorsorra párhuzamos egyenes (kék). Az arra merőlegese piros egyenesek (vetítési sugarak) pedig metszik a képet egy adott t eltolással. Fölötte a vetítősugarak mentén vett vonalintegrálók értékét összesítő görbe jelenik meg. A jobbra a vetítések eredményeiből összeállított szinogram látható, amely a mintavételezés eredményeit összegzi.

Az egyes vetítések eredményét egy mátrixba rendezzük, amelyet szinogramnak nevezünk. A szinogram oszlopai az egyes szögeknél számított vetületek, míg a sorok az adott t távolságoknál kapott vonalintegrál értékeket tartalmazzák a megadott szögnél. A szinogramból megfelelő rekonstrukciós eljárással visszaállítható az eredeti kép egy közelítése, ahol a rekonstrukció pontossága függ a mérési adatoktól és az adott rekonstrukciós algoritmustól.

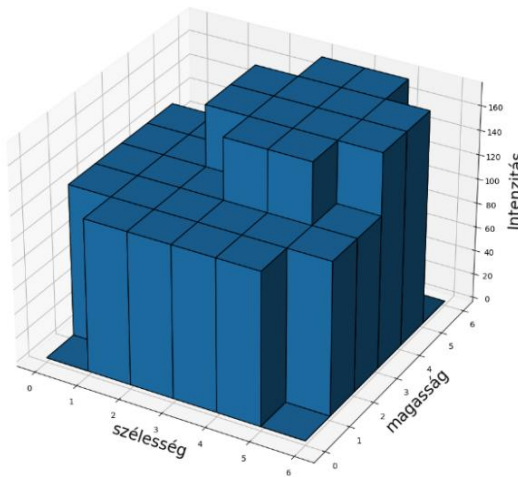
1.4 Diszkrét Radon-transzformáció körfüggvényekkel

Ebben az esetben a vetítendő függvény nem pixeleknél megfeleltethető dobozfüggvények lineáris kombinációjaként áll elő. A bázisfüggvények most körök felett vesznek fel nem nulla értéket. Ezeket a függvényeket három jellemzővel írjuk le. A középpont koordinátájával (C_x, C_y) a sugarával R és a kör fölött a függvény által felvett konstans c_k értékkel, ami az intenzitásának feleltethető meg. Az egyszerűbb jelölés érdekében a továbbiakban eltekintünk a függvény középpontjának és sugarának feltüntetésétől.

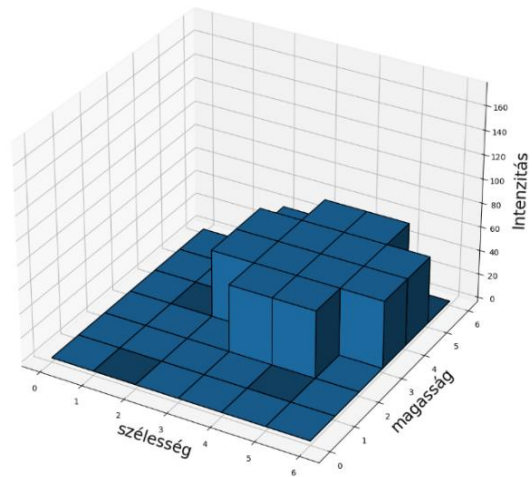
A folytonos jel diszkrét reprezentációjához itt bevezetjük a körfüggvényt, amire a továbbiakban röviden *circ*-ként történik a hivatkozás. A függvényérték egy, ha az adott pont a körön belül helyezkedik el, és nulla, ha azon kívül esik [10].

$$circ_k(x, y) = \begin{cases} 1, & \text{ha } \sqrt{(x - C_x(k))^2 + (y - C_y(k))^2} \leq R \\ 0, & \text{egyébként} \end{cases} \quad (1.13)$$

Eredeti kép 3D ábrázolása



Körfüggvény 3D ábrázolása

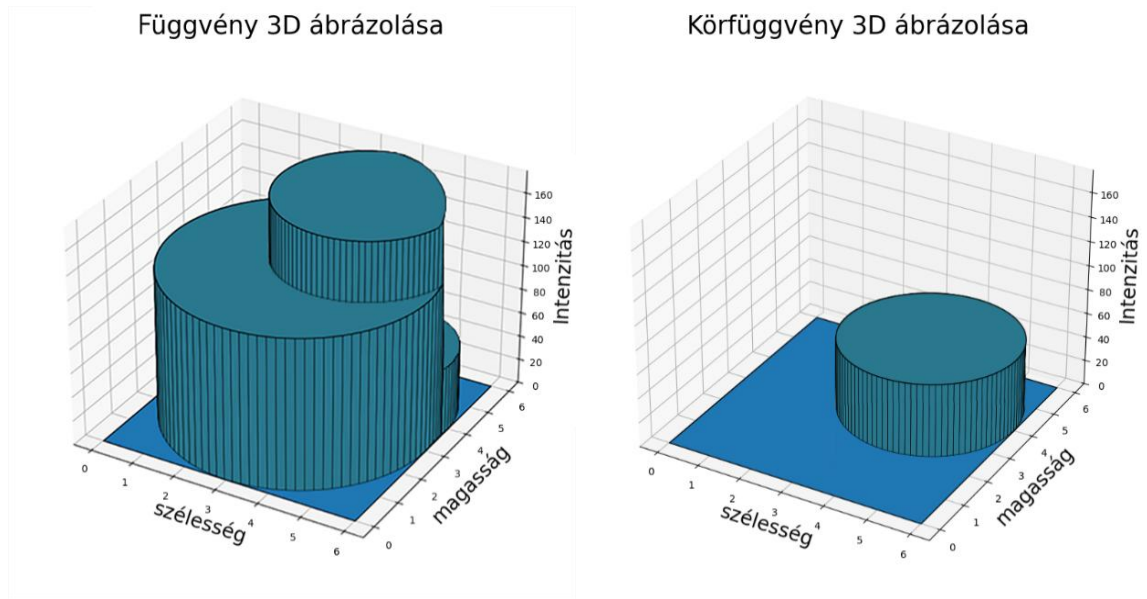


1.4. ábra. Az alábbi ábrán egy tesztkép raszterizált változatai láthatók. A balra a teljes raszterizált kép jelenik meg, míg a jobbra, az egyik raszterizált $circ(x, y)$ bázisfüggvény

Ez a módszer alternatívát kínál a hagyományos pixeles megközelítéssel szemben, mivel a kép leírásához folytonos, kör alakú elemeket használ. Különösen előnyös lehet olyan esetekben, amikor a kép tartalma ívelt, sima határvonalú struktúrákból áll, amelyek nehezen reprezentálhatók négyzetrácsos alapon. Ezen felül lehetővé teszi, hogy a képen

elhelyezkedő körök között lépkedve a diszkrét reprezentációt úgy építjük fel, hogy minden egyes körhöz a középpontját, sugarát és intenzitását rendeljük hozzá.

A cél továbbra is az, hogy a vetítéshez használt egyenes mentén meghatározzuk a vonalintegrált, azaz kiszámítsuk, milyen mértékben járulnak hozzá az egyes objektumok az adott irányú vetülethez. Minden olyan kör esetében, amelyet az adott egyenes metsz, meghatározzuk az áthaladó szakasz hosszát, majd ezt megszorozzuk a körhöz rendelt intenzitásértékkel. Ennek eredményeként a vonalintegrál a körökhöz tartozó intenzitásértékek és az egyenes által átmetszett szakaszok hosszának súlyozott összegeként fejezhető ki.



1.5. ábra. Az alábbi ábrán egy tesztfüggvény olyan változatai láthatók, amelyeknél nem történt raszterizáció. A baloldalon a teljes függvény jelenik meg ebben a formában, amíg a jobboldalon egy kiválasztott körhöz tartozó *circ* bázisfüggvény az együtthatójával súlyozva került kiemelésre.

A Radon-transzformáció kiszámításához ismét kihasználható a linearitás tulajdonsága, vagyis a transzformáció az egyes körfüggvényekre külön-külön alkalmazható, majd az eredményeket összegezzhetjük. Tegyük fel hogy a kép függvénye az alábbi formában adható meg, ahol a k az egyes körfüggvények indexe:

$$f(x, y) = \sum_k c_k \text{circ}_k(x, y) \quad (1.14)$$

Ezt a kifejezést visszahelyettesítve a Radon-transzformációba (1.2), majd minden egyes *circ* bázisfüggvényre alkalmazzuk. Így a teljes függvény Radon-transzformációja az

egyes köralapú bázisfüggvények súlyozott összegeként írható fel:

$$R(f) = \sum_k c_k R(circ_k)(x, y) \quad (1.15)$$

Ebből következik, hogy ha adott (φ, t) paraméterekre kiszámítjuk a Radon-transzformált értéket, akkor a fentebb szereplő összegzésben minden egyes bázisfüggvényhez tartozó integrált külön értékeljük ki. Az integrálok felírásával megkapjuk a következő alakot:

$$R(f)(\varphi, t) = \sum_k c_k \int_{-\infty}^{\infty} circ_k(t \cos(\varphi) - q \sin(\varphi), t \sin(\varphi) + q \cos(\varphi)) dq \quad (1.16)$$

1.5 Körfüggvény vonalintegráljának számítása

A *circ* függvény értéke csak az adott kör tartományában nem nulla, így a q paraméter szerinti integrál valójában csak abban a tartományban ad hozzájárulást, ahol az egyenes metszi az adott kör területét. Emiatt bár az integrálási határok $-\infty$ és ∞ között vannak megadva, a *circ* nullán kívüli értéke csak egy véges tartományra korlátozódik. Ennek következtében a tényleges integrálási tartományt az adott köralak határolja be, így a végtelen határok pusztán formálisak és a számítás során, a körön kívüli részek nem járulnak hozzá az összeghez.

A számítás alapját tehát az képezi, hogy minden egyes körfüggvény esetén meghatározzuk a vetítősugar és a körfüggvény metszéspontjait, majd az áthaladó szakasz hosszát az intenzitással súlyozzuk.

Ennek folyamata a vetítősugar normálvektorának ismeretével kezdődik, ami már ismert (1.7). Ezt felhasználva meghatározható az egyenes irányvektora, egy $\frac{\pi}{2}$ szögű forgatás segítségével. Ehhez a forgató mátrix alakja az alábbi:

$$\mathbf{R}_{\frac{\pi}{2}} = \begin{bmatrix} \cos\left(\frac{\pi}{2}\right) & -\sin\left(\frac{\pi}{2}\right) \\ \sin\left(\frac{\pi}{2}\right) & \cos\left(\frac{\pi}{2}\right) \end{bmatrix} \quad (1.17)$$

A mátrixművelet egy új vektort eredményez, amely a normálvektorral derékszöget zár be és az egyenessel egyállású:

$$\mathbf{r} = \mathbf{R}_{\frac{\pi}{2}} \cdot \mathbf{n} = \begin{bmatrix} -\sin(\varphi) \\ \cos(\varphi) \end{bmatrix} \quad (1.18)$$

Ezután az így kapott vektor komponenseit külön-külön is felírjuk, mivel a későbbi lépésekben felhasználásra kerülnek.

$$r_x = -\sin(\varphi) \quad , \quad r_y = \cos(\varphi) \quad (1.19)$$

A metszéspontok pontos meghatározása érdekében figyelembe kell vennünk a kör egyenletét, amely az alábbi általános alakban fejezhető ki. Ez az egyenlőség meghatározza a síkon elhelyezkedő összes olyan pontot, amelyek egy adott (C_x, C_y) középponthoz képest egy rögzített R sugarú távolságra helyezkednek el:

$$(x - C_x)^2 + (y - C_y)^2 = R^2 \quad (1.20)$$

A következő egyenletben (x, y) koordináták az egyenes olyan pontjait jelölik, amelyek lehetséges metszéspontokként szolgálhatnak. Az egyenes minden pontja paraméteresen írható fel, viszont csak azok a pontok lehetnek valódi metszéspontok, amelyek egyidejűleg teljesítik a kör egyenletét is, ennek megfelelően az x koordináta a következőképpen fejezhető ki:

$$x = v_{0x} + q r_x \quad (1.21)$$

Ugyanígy módon adhatjuk meg az y koordinátát is, mely a megadja a pont függőleges elhelyezkedését, és amelyet az alábbi kifejezés ír le:

$$y = v_{0y} + q r_y \quad (1.22)$$

Eszerint a $\mathbf{v}_0$ az egyenes egy referenciapontja, a q paraméter pedig egy valós szám. Ezek alapján az (1.21), (1.22) az (1.20)-ba behelyettesítve állítja elő a következő egyenletet:

$$(v_{0x} + q r_x - C_x)^2 + (v_{0y} + q r_y - C_y)^2 = R^2 \quad (1.23)$$

A továbbiakban a számítások egyszerűsítését és a kód hatékonyabb átláthatóságának növelését szolgálja, ha már a paraméterezés megkezdése előtt kiszámítjuk a kör középpontja (C_x, C_y) és a vetítendő sugár referenciapontja (v_{0x}, v_{0y}) közötti eltolás koordinátáit:

$$k_x = (v_{0x} - C_x) \quad , \quad k_y = (v_{0y} - C_y) \quad (1.24)$$

Ezeket az értékeket visszahelyettesítve az előző egyenletbe, az alábbi átalakított alakhoz jutunk:

$$(k_x + q r_x)^2 + (k_y + q r_y)^2 = R^2 \quad (1.25)$$

Ez az egyenlet azt fejezi ki, hogy a vetítősugár paraméteres egyenlete milyen feltételek mellett metszi a kört. Annak érdekében, hogy az egyenlet további feldolgozása egyszerűbb legyen, felbontjuk a négyzetre emeléseket, így kapjuk a következő alakot:

$$k_x^2 + 2 (k_x q r_x) + (q r_x)^2 + k_y^2 + 2 (k_y q r_y) + (q r_y)^2 = R^2 \quad (1.26)$$

Az egyenlet átrendezésével és a tagok q szerint történő csoportosításával egy másodfokú egyenletet kapunk. Ebben az alakban már jól láthatóak a másodfokú egyenlet együtthatói, amelyeket az a , b és c jelöl:

$$q^2(r_x^2 + r_y^2) + q(2(k_x r_x) + 2(k_y r_y)) + (k_x^2 + k_y^2 - R^2) = 0 \quad (1.27)$$

Ezek az együtthatók a következőképpen írhatók fel:

$$a = r_x^2 + r_y^2 \quad (1.28)$$

$$b = 2(k_x r_x + k_y r_y) \quad (1.29)$$

$$c = k_x^2 + k_y^2 - R^2 \quad (1.30)$$

A megoldáshoz vezető következő lépésként szükséges kiszámítani a másodfokú egyenlet diszkriminánsát, ami a következő képlettel adható meg:

$$\Delta = b^2 - 4ac \quad (1.31)$$

A diszkrimináns értéke meghatározza a vetítősugár és a körfüggvény metszéspontjainak számát. Abban az esetben, ha negatív, nem kapunk metszéspontot, mivel az egyenletnek nincs valós megoldása, ha nulla, akkor az egyenes érinti a kört, vagyis egyetlen közös pontjuk van, ha pedig pozitív, akkor az egyenlet két valós gyökkel rendelkezik.

A metszéspontok számának meghatározásával a következő képletek alapján kapjuk meg a metszéspont paraméteres értékeit:

$$q_1 = \frac{-b - \sqrt{\Delta}}{2a} \quad , \quad q_2 = \frac{-b + \sqrt{\Delta}}{2a} \quad (1.32)$$

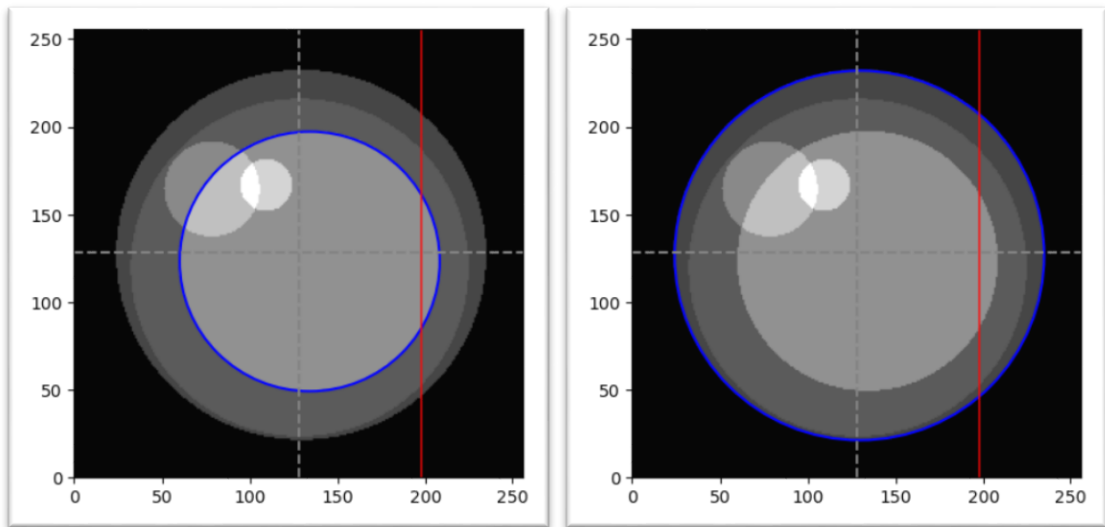
Ezek az értékek a másodfokú egyenlet gyökei. A metszéspontok koordinátái úgy számíthatók ki belőlük, ha behelyettesítjük ezeket az egyenes paraméteres egyenletébe, így a következő egyenletet kapható:

$$\mathbf{v}_1 = \mathbf{v}_0 + q_1 \mathbf{r} \quad , \quad \mathbf{v}_2 = \mathbf{v}_0 + q_2 \mathbf{r} \quad (1.33)$$

Ebben az összefüggésben a $\mathbf{v}_1$ és $\mathbf{v}_2$ a vetítősugar és a kör metszéspontjainak pozíció vektorait jelölik. Ezek ismeretében kiszámítható az a távolság, amelyet a vetítősugar a kör belsejében tesz meg. A távolság az alábbi képlettel adható meg:

$$L_i = \sqrt{(v_{1x} - v_{2x})^2 + (v_{1y} - v_{2y})^2} \quad (1.34)$$

Az L_i mennyiség fejezi ki azt a tényleges hosszúságot, amely az egyenes és a kör metszetének hossza, ami alapjául szolgál a vonalintegrál számításának.



1.6. ábra. A vonal és a vetítősugar metszéspont vizsgálata, ahol a piros vonal a vetítősugarat, a kék kör pedig azt a kört reprezentálja aminél a metszésszakasz számítása zajlik.

Ezt követően a feldolgozási lépések megegyeznek a *rect* bázisfüggvényeket

feltételező eljárásával, vagyis a vetítési sugár paraméterezéséhez itt is az irányszöget és az eltolási távolságot használjuk. A különbség abban áll, hogy ebben az esetben nem négyzet alakú pixeleken, hanem kör alakú tartományokon hajtjuk végre a számítást. Ez különösen alkalmas lehet olyan alkalmazásokhoz, ahol a képi információ nem rácspontokhoz kötött, hanem inkább természetesebb, elmosódottabb objektumokból áll.

2. fejezet

Alkalmazott eszközök

2.1 Python és Python csomagok

A szoftveres alapot a Python programozási nyelv biztosítja, amely könnyen olvasható szintaxisa és dinamikus típuskezelése mellett a gazdag könyvtárkínálattal is hozzájárul a hatékony fejlesztéshez. A Conda virtuális környezettel a megvalósított program függőségeit könnyen kezelhetjük, így a kódbázis birtokában a mérések reprodukálhatóak.

A numerikus számítások gerincét a NumPy csomag adja, mely homogén, memóriában folyamatosan tárolt tömböket biztosít, és vektorizált, C-ben megírt függvények révén nagyságrendekkel gyorsabb műveleteket tesz lehetővé a python listákhoz képest [9]. A tömbszeletelés és lineáris algebrai függvénykönyvtár egyaránt hozzájárult a diszkrét Radon-transzformáció megvalósításához.

Az eredmények szemléletes megjelenítését és összehasonlítását a Matplotlib [6] tette lehetővé, amely parancssori szkriptekbe ágyazott hívásokkal rajzolt akár szinogramokat vagy a rekonstruált metszeteket. A grafikonok és képek generálása egyszerű függvényhívásokkal történt, így a különböző módszerek eredményei közvetlenül összehasonlíthatóvá és jól dokumentálhatóvá váltak.

A TomoPy egy tomográfias adatok feldolgozására szolgáló könyvtár, amelyet a *rect* illetve a *circ* módszerek összehasonlítására lett felhasználva. A könyvtár különféle előfeldolgozási és rekonstrukciós eljárásokat kínál, amelyek lehetővé teszik a tomográfiai adatok különböző szintű feldolgozását [8],[11]. A TomoPy könyvtárból elsősorban a *Tomopy.project* függvény került alkalmazásra a szinogramok előállítására, vagyis az eredeti képek vetületeinek létrehozására. Ezen felül a *tomopy.recon* függvény a szinogramokból történő képrekonstrukció megvalósítására lett alkalmazva.

Ezek a komponensek együttesen olyan egységes keretrendszert alkottak, amelyben a numerikus eljárások, a megjelenítés és a referenciaeljárások zökkenőmentesen illeszkedtek egymáshoz, a kód moduláris felépítésének köszönhetően

pedig minden részfunkció könnyen cserélhető és tesztelhető maradt.

2.2 Numba CUDA – párhuzamos számítás

A nagy méretű tomográfiás adatok feldolgozása során a szekvenciális Python kód futásideje korlátot jelent, ezért volt szükség a Numba csomagra és annak CUDA integrációjára. A Numba dekorátorokkal jelölt függvényeit futásidőben gépi kódra fordítja, így a kritikus számítási magokat hatékonyan lehet párhuzamosítani [5].

A CUDA környezettel összekapcsolódva lehetőség nyílt arra, hogy a *@cuda.jit* dekorátorral ellátott kernelfüggvényeket a GPU segítségével fussanak, a blokk és rácsstruktúrák definíciója, a globális és helyi memóriaterületek kezelése, valamint a GPU és CPU közötti adatmozgatás mind a Numba futásidejű rendszerén keresztül történik. Ennek révén a Radon-transzformáció egyes lépései nem csupán szálak mentén, hanem többszáz GPU magon futnak le. Így fokozott csökkenést érhetünk el a számítási időben a nagy felbontású képi adatok esetén. A CUDA memóriamásolási mechanizmus megoldja, hogy a NumPy tömbök és a GPU memória közötti átvitel átlátható legyen, így a kernelhívások közötti szinkronizáció egyszerűbb [12].

A GPU párhuzamosítás integrálása nem csak a futási idő optimalizálásához járul hozzá, de megmutatja, hogy a Python alapú fejlesztés rugalmassága és a GPU hatékonyság kombinálható egyetlen környezetben, így a tomográfiás rekonstrukciók kísérleti és összehasonlító vizsgálatai során mind a pontosság, mind a nagy adathalmazokra skálázódó teljesítmény a lehető legmagasabb szinten marad.

3. fejezet

A program felépítése

3.1 Mappaszerkezet

Az alábbi rész a projekt mappaszerkezetének felépítését mutatja be. A struktúra célja, hogy logikus és jól átlátható módon rendszerezze a fájlokat és könyvtárakat, ezzel megkönnyítve a projektben való tájékozódást, valamint a fejlesztési és feldolgozási munkafolyamatok átláthatóságát.

A *radon_rect* mappában a diszkrét dobozfüggvény alapú módszer megoldás implementációja található. Itt a számítások külön CPU és GPU alapú megközelítéssel valósulnak meg. A *cpu* almappában található szkript nemcsak a szinogram generálását végzi el, hanem a működés vizualizálására külön képeket is készít, lehetővé téve a részletes elemzését. Az eredményeket TIFF és PNG formátumban menti el közvetlenül a szkript mellett lévő *results* mappába. A *cpu* mappa mellett megtalálható a *gpu* mappa is, ami a CUDA alapú gyorsított változatát tartalmazza, amely a számításokat a GPU erőforrásait kihasználva hajtja végre.

A *radon_circ* mappa tartalmában a körfüggvények segítségével történő szinogram generálás zajlik. A fájlstruktúra itt is hasonló, egy *cpu* mappa tárolja a processzor alapú megoldást, míg mellette egy másik almappában a GPU implementáció is rendelkezésre áll. Az itt generált szinogramok és az eredmény képek itt is elmentésre kerülnek, mint a dobozfüggvény alapú módszer esetében, lehetővé téve az azonnali vizuális összehasonlítást.

A *radon_tomopy* mappában a TomoPy könyvtár hívó szkriptje található. Amely könyvtár a különböző módszerek összehasonlításához és teszteléséhez lett felhasználva. Az elkészült eredményeket itt is TIFF formátumban menti el a mappán belül a *results* mappába.

A *test_images* mappa az elemzésekhez használt, generált képeket tartalmazza, a hozzájuk tartozó leíró TXT fájlok pedig a szomszédos *txt_files* mappában találhatók. A

TXT fájlok tömbök formájában tárolják a köradatokat, ahol minden elem egy-egy kört reprezentál (középpont (x, y), sugár és intenzitás). Ezek az adatok szolgálnak a további számítások, tesztek és kiértékelések alapjául.

A *image_generator* mappában található azok a szkriptek, amelyek a tesztképeket generálják, CPU és GPU alapú implementációval, külön mappákba rendezve. Emellett itt található olyan segédszkript is, amely megszámlolja, hogy egy adott képen hány olyan pixel található, amelynek intenzitása nem nulla. A képek PNG formátumban is elérhetők a mappában, amin látszik a maximális intenzitás értéke és a kép mérete, míg a generált adatok végül a *test* mappába kerülnek TIFF formátumban.

A *reconstruction_tomopy* mappában található a TomoPy rekonstrukciós függvényének hívása és annak alkalmazása. Ez a szkript a szinogramokból visszaállítja az eredeti kép egy közelítését, lehetővé téve az eljárások pontosságának ellenőrzését és az egyes megoldások összehasonlítását.

A *tests* mappa a szinogramok és a rekonstruált képek teszteléséhez szükséges szkripteket tartalmazza, külön mappákba szervezve. Ezek a szkriptek lehetővé teszik a különböző rekonstrukciós módszerek objektív összehasonlítását és értékelését, különböző statisztikai hibamértékek, például a Mean Squared Error (MSE) (4.1), Mean Absolute Error (MAE) (4.2) és Maximum Absolute Error (MaxAE) (4.3) kiszámításán keresztül.

A mért értékek alapján a szkriptek hisztogramokat és eltérésképeket is generálnak, amelyek vizuálisan is szemléltetik a különbségeket. Ezeken a hisztogramokon jól nyomon követhető, hogy adott módszer milyen mértékű eltérést produkál az eredeti képhez képest.

Továbbiakban a rendszer létrehoz különbségképeket is TIFF formátumban, ahol a rekonstruált és az eredeti képek közötti eltérések szürkeárnyaltos formában vizuálisan is összehasonlíthatók. Így nemcsak numerikus, hanem képi szinten is értékelhető a rekonstrukció minősége.

A szkriptek a tesztelendő képeket és fájlokat a megfelelő mappákból olvassák be, és az eredményeket is a kijelölt helyekre mentik. Ennek köszönhetően elegendő csupán a kép nevét megadni, a teljes elérési út megadása nélkül.

Az egységes és reprodukálható kimeneti formátum biztosítása érdekében a rendszer a TIFF fájlformátumot alkalmazza, ami egy veszteségmentes képformátum, így ideális tudományos és orvosi képfeldolgozáshoz. Ezzel biztosítható, hogy az eredeti adatok torzítás nélkül kerüljenek mentésre és további feldolgozásra.

3.2 Bázisfüggvény választásától független eljárások

Az alábbi kódrészlet a *calculate_radon_transform* függvényt ismerteti, amely mind a *rect*, mind a *circ* bázisfüggvények esetén ugyanazon alapvető logikát alkalmaz, mivel a transzformáció mindkét esetben azonos elven működik. Vagyis ugyanúgy megtörténik a szögek szerinti forgatás és a vetítendő sugár alapvető számításai.

Nagy vonalakban először meghatározza a szinogram méretét, majd iteratív módon végigiterál az összes sugárpályán. Minden egyes forgatási szög és eltolási érték esetén meghatározza a sugár és a bázisfüggvény tartójának metszéspontjait, kiszámítja ezek távolságát, és az így kapott intenzitásokkal súlyozott értékeket folyamatosan összegezi. A folyamat végén egy teljes szinogramot ad vissza.

```
1  def calculate_radon_transform(...):
2      divisor = np.floor((side_length * np.sqrt(2) - side_length + 4) / 2)
3      num_columns = (divisor * 2 + side_length)
4
5      sinogram = np.zeros((int(num_columns), len(angles_degrees)),
6                          dtype=np.float64)
7
8      # A vetítősugár szögek szerinti forgatása.
9      for angle_idx, angle in enumerate(angles_degrees):
10         angle_rad = np.radians(angle)
11
12         # A vetítősugár léptetése az x tengelyen.
13         for offset_idx, line_x_offset in enumerate(np.arange(0.5 - divisor,
14                                                                side_length + divisor)):
15
16             # A vetítősugár koordinátáinak számítása a képközéppont,
17             # forgatás és eltolás alapján.
18             line_center = np.array([
19                 side_length / 2 - (side_length / 2 *
20                 (np.sin(angle_rad))) +
21                 (line_x_offset * (np.sin(angle_rad))),
22                 side_length / 2 + (side_length / 2 *
23                 (np.cos(angle_rad))) -
24                 (line_x_offset * (np.cos(angle_rad)))],
25                 dtype=np.float64)
26
27             line_integral = 0
28
29             # Rect illetve circ bázisfüggvények végigiterálása.
30             ...
31                 intersection_points = []
32
33                 num_intersections = calculate_line_intersections(...)
34
35                 distance = calculate_max_distance(intersection_points)
36
37                 line_integral += intensity * distance
38
39             sinogram[offset_idx, angle_idx] = line_integral
40
41     return sinogram
```

3.1. ábra. A *rect* és a *circ* bázisfüggvények azonos működésű kódrésze.

A fenti eljárás elsőként meghatározza a szinogram/mátrix méretét, amely a kép dimenzióiból, a vizsgált szögek számából és a *divisor* változóból adódik. A *divisor* a négyzet alakú kép teljes átlójának bejárását biztosítja, amelyet az oldalhossz $\sqrt{2}$ -es

szorzata határozza meg.

A megfelelő méretű NumPy tömb inicializálása után iterálunk a megadott szögértékeken, majd minden egyes szögnél a vetítési sugarakat a $-divisor$ és képméret $+ divisor$ tartományban léptetjük. A $0.5 offset$ biztosítja, hogy a vetítősugarak ne essenek egybe pixel határoló egyenesekkel. Az iteráció során kiszámítjuk az elforgatott és eltolással módosított egyenes x tengelyen lévő v_0 pontját, ami a 3.1 ábra 18. sorában látható.

A következő lépés a bázisfüggvények számítása, amelynek részletes kifejtésére a következő két alfejezetben kerül sor. A számítások alapján meghatározhatók a bázisfüggvények tartóinak és a sugár metszéspontjai. Ez a `calculate_..._line_intersections` függvényen belül történik, ahol a „...” helyére a *rect* vagy *circ* kifejezések kerülhetnek, a választott bázisfüggvény típusától függően. Az eredményül kapott metszéspontok segítségével a `calculate_max_distance` függvény kiszámíthatja a metszés hosszát. A sugár által érintett pixelek vagy körök intenzitásértékeit a metszés hosszával megszorozva állnak elő azok az értékek, amelyeket összegezve megkapjuk a vetítési értéket a szinogram/mátrix adott pozíciójában, vagyis az $R(f)(\varphi, t)$ -t.

3.3 Dobozfüggvény tartója és a vetítősugár közti metszések számítása

A `calculate_rect_line_intersections` függvény célja, hogy meghatározza egy adott irányú egyenes és egy pixel oldalainak (pontosabban azok meghosszabbított oldalegyeneseinek) metszéspontjait, majd azt egy tömbben tárolja.

A függvény a bemeneti paraméterek alapján kiszámítja, hogy az adott sugár a pixel melyik oldalegyeneseit metszi. Majd a talált metszéspontokat ellenőrzi, hogy az valóban a pixelél tartományába esik-e vagy annak meghosszabbított egyenesénél található. Az eredménytől függően adja vissza a metszéspontok koordinátáit, amik ez alapján valóban a pixelélen fognak elhelyezkedni.

Az egyenes irányát `pixel_edge_direction` irányvektor határozza meg. Az erre merőleges vektor (`pixel_normal`) a `rotation_90_matrix` (1.17) segítségével számítjuk ki. Az egyenes és a pixel oldalegyenese közötti metszéspont meghatározásához a függvény először kiszámít egy egyenletrendszerhez szükséges mátrixot és jobb oldali vektort. Az $\mathbf{N}$ (a kódban $\mathbf{A}$) mátrix és a $\mathbf{w}$ (a kódban $\mathbf{b}$) vektor az (1.11)-es egyenletben került kifejtésre. Ezek alapján egy lineáris egyenletrendszer áll elő (1.10), ez a művelet a 3.2

ábra 23. sorában látható. Az egyenletrendszer megoldására a `np.linalg.solve` függvény szolgál. Ha az N determinánsa nullához közeli értéket vesz fel, akkor a két egyenes párhuzamos és nincs megoldás. Az az eset, amikor végtelen sok metszéspont keletkezik, vagyis a sugár egyenese teljesen egybeesik a pixel meghosszabbított élének egyenesével, a 0.5-ös pixelközépre tolásnak köszönhetően kizárható. A feles eltolást jól bemutatja az 1.2 ábra ahol látható, hogy a piros vonal, ami a vetítősugár, pontosan a pixel felén halad át. (Az 1.2. ábrahoz képest egy eltérő koordináta rendszer kerül alkalmazásra, de az eredmények a matematikai levezetésben kapottakkal megegyeznek.)

```

1  def calculate_rect_line_intersections(angle_rad_pixel, pixel_corner, line_center, pixel,
2                                     intersection_points, line_normal, rotation_90_matrix):
3
4
5      pixel_edge_direction = np.array([np.cos(angle_rad_pixel)],
6                                     [np.sin(angle_rad_pixel)]], dtype=np.float64)
7
8      pixel_normal = np.dot(rotation_90_matrix, pixel_edge_direction)
9
10     # Lineáris egyenletrendszer összeállítása ( Ax = b )
11     A = np.concatenate((line_normal.T, pixel_normal.T), axis=0)
12
13     b = np.array([np.dot(line_normal.T, line_center).item(), np.dot(pixel_normal.T,
14                             pixel_corner).item()] )
15
16     epsilon = 1e-6
17
18     # Ha a vetítősugár és a pixel egyenese párhuzamos, vagyis nincs metszéspont,
19     # akkor az A mátrix determinánsa nullához közeli értéket vesz fel.
20     if abs(np.linalg.det(A)) < epsilon:
21         return
22
23     V = np.linalg.solve(A, b).reshape(2, 1)
24
25     # Ellenőrzés hogy a pixel határokon belül legyen a talált metszéspont.
26     if np.isclose(angle_rad_pixel, np.pi / 2): # függőleges él.
27         if (pixel[1] - epsilon) <= V[1] <= (pixel[1] + 1 + epsilon):
28             intersection_points.append(V)
29
30     elif np.isclose(angle_rad_pixel, np.pi): # vízszintes él.
31         if (pixel[0] - epsilon) <= V[0] <= (pixel[0] + 1 + epsilon):
32             intersection_points.append(V)

```

3.2. ábra. Kódrészlet, ami a vetítősugár és a pixel metszéspontjait számolja ki. A pixel négy oldalának meghosszabbított egyenesén külön-külön vizsgálja, hogy található-e metszéspont.

Végül a kiszámított metszéspont (1.12) validálása történik. Ahol eldől, hogy a pixel határain belül helyezkedik-e el a metszéspont. Ekkor kiderül, hogy az adott irányú egyenes ténylegesen metszi-e a pixel adott határoló szakaszát. Ha a feltételek teljesülnek, a metszéspont bekerül az `intersection_points` listába.

3.4 Körfüggvény tartója és a vetítősugár közti metszések számítása

A `calculate_circ_line_intersections` függvény célja, hogy meghatározza egy adott kör és egyenes metszéspontjait kétdimenziós térben. A bemenetként kapott kör középpontja és

sugara, valamint a sugárként értelmezett egyenes iránya és helyzete alapján a függvény kiszámítja, hogy léteznek-e metszéspontok, és ha igen, pontosan hol helyezkednek el.

Első lépésként az egyenes irányvektorát számolja ki. Ezután meghatározza a kör középpontja és az egyenes egy adott pontja közötti távolságát (1.21),(1.22). A metszéspontok meghatározásához a kör egyenletét (1.20) és az egyenes paraméteres egyenletét kombinálva egy másodfokú egyenlet (1.27) áll elő. A függvény kiszámítja a másodfokú egyenlet együtthatóit (1.28),(1.29),(1.30) amik a kódban a 3.3 ábra 12-14. sorában helyezkednek el. Majd a diszkrimináns értékét ellenőrzi (1.31). Abban az esetben, mikor a diszkrimináns negatív értéket vagy nullát vesz fel, akkor az egyenes és a kör nem metszi, vagy csak érintik egymást, így a függvény kilép nulla értékkel.

```

1      def calculate_circ_line_intersections(circle_center, circle_radius, angle_rad_line,
2                                          line_center, intersection_points):
3
4
5          line_direction = np.array([np.cos(angle_rad_line), [np.sin(angle_rad_line)]],
6                                     dtype=np.float64)
7
8          dx, dy = line_direction.flatten()
9          fx, fy = line_center.flatten() - circle_center.flatten()
10
11         # Másodfokú egyenlet együtthatóinak számítása.
12         a = dx**2 + dy**2
13         b = 2 * (fx * dx + fy * dy)
14         c = fx**2 + fy**2 - circle_radius**2
15
16         # Gyök alatti rész ( diszkrimináns ) számítása.
17         discriminant = b**2 - 4 * a * c
18
19         # Ha a diszkrimináns negatív értéket vesz fel, nincs metszéspont.
20         if discriminant <= 0:
21             return 0
22
23         # Lehetséges metszéspontok kiszámítása.
24         sqrt_discriminant = np.sqrt(discriminant)
25
26         t1 = (-b - sqrt_discriminant) / (2 * a)
27         t2 = (-b + sqrt_discriminant) / (2 * a)
28
29         num_intersections = 0
30
31         for t in [t1, t2]:
32             V = line_center + t * line_direction
33             intersection_points.append(V)
34             num_intersections += 1
35
36         return num_intersections

```

3.3. ábra. Kódrészlet, ami a vetítősugar és a rect metszéspontjait számolja ki másodfokú egyenlet megoldásával.

Pozitív diszkrimináns esetén a másodfokú egyenlet megoldásából meghatározza a q_1 és q_2 paramétereket (1.32) 3.3. ábra 26-27. sorában, amelyek az egyenes azon pontjait adják meg, ahol a metszés történik. Végül ezeket a pontokat kiszámítja (1.34) és eltárolja az *intersection_points* listában.

3.5 Egyéb működésbeli eltérések

A *rect* és *circ* bázisfüggvények implementációja között nemcsak a geometriai szempontból van különbség, hanem a feldolgozás logikája és optimalizálása is eltér.

A *rect* esetben a gyorsítás érdekében a Bresenham algoritmus [4] is alkalmazásra kerül, amely hatékonyan meghatározza az adott vetítősugar környékén lévő pixeleket. Így nem szükséges minden vetítősugarra végigiterálni a teljes képen, csak azokra a pixelekre koncentrálni, ahol ténylegesen várható metszés.

A *circ* függvénynél ezzel szemben először a bázisfüggvény tartója és a vetítősugar közti metszés számításának előfeltételeként ellenőrzi a diszkrimináns értékét. Ha az negatív, az adott vetítősugar biztosan nem metszi a kört, így a további számítás elhagyható. Ez a célzott szűrés jelentősen csökkenti a szükséges műveletek számát.

3.6 GPU megvalósítás

A GPU alapú Radon-transzformáció célja a számítási sebesség jelentős növelése a CPU-s megoldáshoz képest, különösen nagy képméret vagy sok szög esetén. Működését tekintve a GPU-s megvalósítás a CPU-s algoritmussal logikailag azonos eredményt szolgáltat. Végigiterál a megadott szögeken és eltolásokon, meghatározza a vetítősugar és a körök metszéspontjait, majd ezek távolságát súlyozza a kör intenzitásával, és eltárolja az értékeket a színogramban.

A fő különbség a párhuzamosítás során hogy miként kerül elosztásra a művelet a szálak között (mivel itt a CPU-val ellentétben nem sorban haladunk) és az adattípusok kezelésében rejlik. A GPU-s változatban minden vetítősugar (szög és eltolás tekintetében) külön szálként kerül kezelésre, így a számítási feladat erősen párhuzamosítható. Ehhez szükséges a CUDA technológia használata, amely külön felkészülést igényel, például az adatok másolását a GPU memóriájába, illetve olyan, *@cuda.jit* dekorátorral ellátott függvényeket, amelyek a GPU-n is futtathatók [12].

A GPU-s kódban külön figyelmet kell fordítani a memóriakezelésre és az adattípusokra, például a NumPy struktúrák helyett *cuda.local.array* típusokat használni, és kerülni kell a beépített függvényhívásokat vagy nem támogatott NumPy műveleteket.

A GPU-s megvalósítás legjelentősebb eltérése a *calculate_radon_transform* kernelfüggvény szerkezetében mutatkozik meg, ahol a CPU-s külső ciklusok (szög és eltolás szerint) egy kétdimenziós CUDA rácásra lettek leképezve. Minden CUDA szál egy

adott (szög, eltolás) kombinációhoz tartozó vetítősugár vonalintegrálját számítja ki, teljesen függetlenül a többitől.

Ahogy a 3.4. ábra látható, a CUDA alapú megvalósítás felépítésében eltér a CPU-s változattól, elsősorban a párhuzamos feldolgozás és az adatszerkezetek kezelése miatt. Ennek ellenére a két változat működési logikája, tehát az algoritmus lépései és az alkalmazott matematikai műveletek alapvetően megegyeznek.

```
1     def calculate_radon_transform(...):
2         angle_idx, offset = cuda.grid(2)
3
4         if angle_idx >= angles_degrees.shape[0] or offset >= (side_length + divisor * 2):
5             return
6
7         angle = angles_degrees[angle_idx]
8         angle_rad_line = math.radians(angle)
9
10        line_x_offset = offset + 0.5 - divisor
11
12        ...
13
14        sinogram[offset, angle_idx] = projection_sum
```

3.4. ábra. Minden CUDA szál egy adott szög és eltolás kombinációhoz tartozó vetítősugár vetületét számolja ki, majd az így kapott összeget a szinogram megfelelő pozíciójába menti.

Viszont kisebb képméreteknél vagy apróbb adatmennyiségeknél a GPU használata lehet lassabb, mivel minden egyes kernelhívásnak és adatmozgatásnak van egy késleltetése. A programnak először fel kell töltenie a bemeneti adatokat a videomemóriába, majd elindítania a GPU-n a számítást, és végül vissza kell másolnia az eredményeket a rendszer memóriájába. Ezek a lépések együtt akár több száz mikromásodpercet is igénybe vehetnek. Ezzel szemben a CPU-nál a függvényhívás és a memóriaművelet általában nanomásodpercek alatt lezajlik, így ha csupán néhány száz vagy ezer kör/pixel metszéspontot kell kiszámolni, a GPU előkészítési költségei dominálnak a tényleges számítási idő fölött.

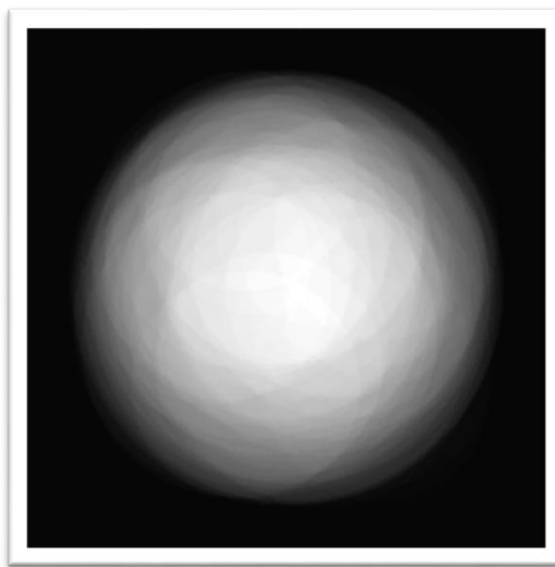
4. fejezet

Mérési eredmények

4.1 Néhány gondolat az eredmények kapcsán

Érdemes megemlíteni, hogy a vizsgálat nem azt méri, hogy a *circ* bázisfüggvény feltételezése mellett készített szinogramok jobb eredménnyel rekonstruálhatóak-e, hanem azon van a hangsúly, hogy kiderüljön, milyen eltérésekre számíthatunk egyik és másik függvénybázis feltételezése esetén.

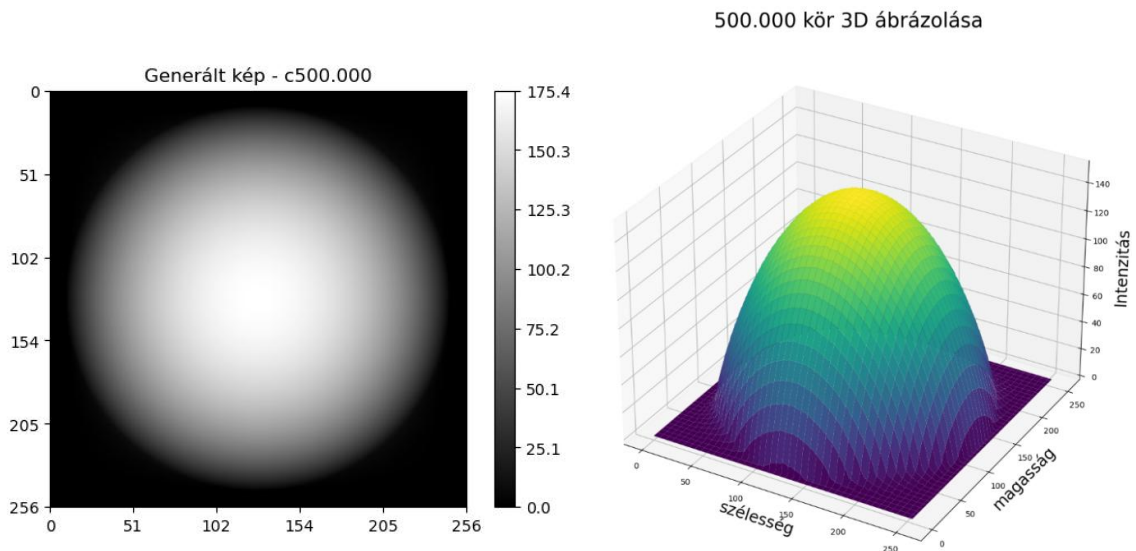
A tesztek és azokból származó eredmények a lentebb látható képhez hasonló képek alapján készültek. A lenti kép egy 256×256 képpont felbontású, szürkeárnyaltos kép, amelyen összesen 101 lebegőpontos paraméterekkel (középpont, sugár, intenzitás) definiált kör helyezkedik el.



4.1. ábra. Egy 256×256 -os tesztkép, ami 101 különböző paraméterezésű kört ábrázol.

Érdekes jelenség figyelhető meg, amikor nagyszámú, például 500.000, véletlenszerűen generált lebegőpontos paraméterekkel rendelkező kört helyezünk

egymásra egy adott térben. Ebben az esetben ezek a körök a megfelelő paraméterezés mellett a képen fokozatosan egy szabályos gömbformát rajzolnak ki. Ez nemcsak esztétikailag látványos, hanem matematikailag is érdekes, mivel a kapott alakzat következtetni enged a határeloszlás szabályos alakjára.



4.2. ábra. Körök száma: 500.000, kép méret: (256, 256), minimin sugárérték = 10, kerettől mért minimális távolság = 5 egység.

4.2 Szinogram vizsgálata

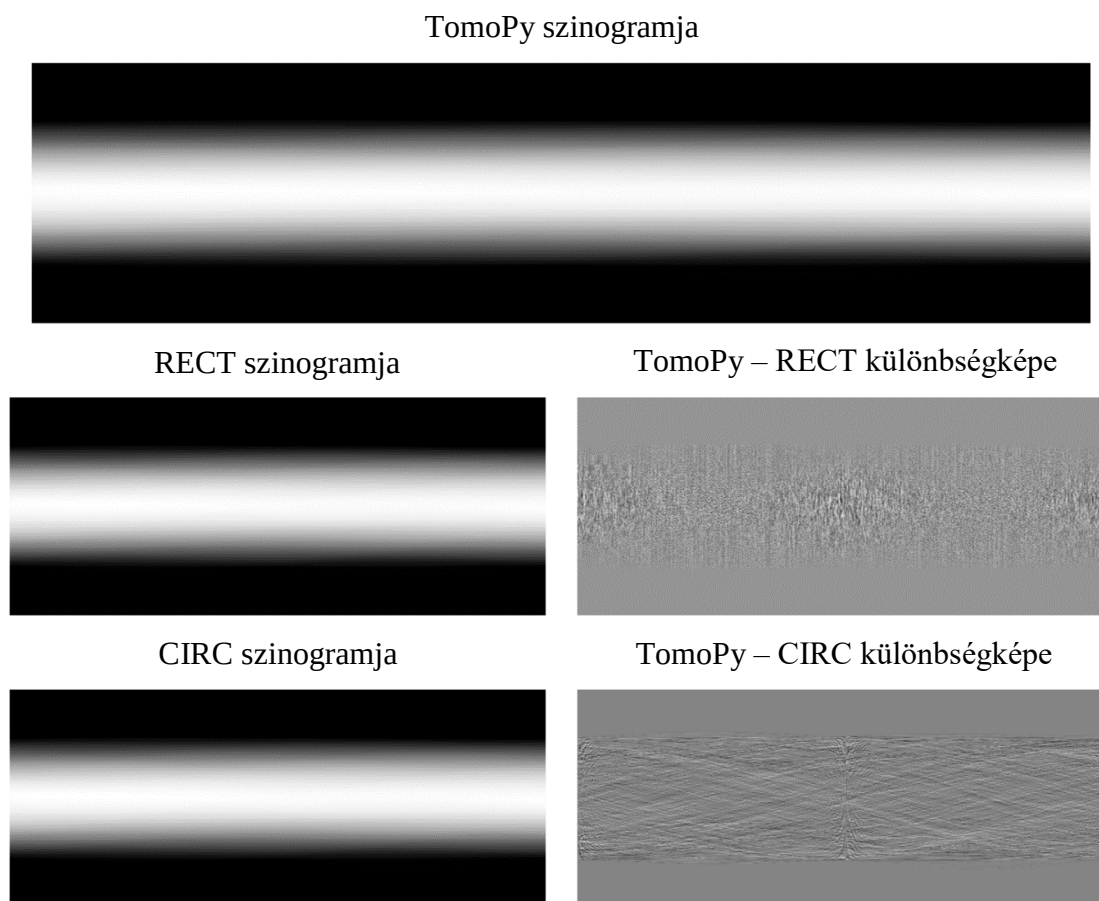
Az összehasonlítás során a módszerek közötti különbségek vizsgálatának egyik hatékony módja az, ha az egyes szinogramokat pixelenként kivonjuk egymásból. Ezzel a módszerrel olyan képet kapunk, amely jól érzékelteti, hogy az egyes technikák által előállított szinogramok hol és milyen mértékben térnek el egymástól. Az így kapott különbségképek alapján egyértelműen megállapítható, hogy mely területeken jelentkeznek a legnagyobb eltérések.

Bár látszólag nem mutatkoznak jelentős különbségek a három szinogram között, abban az esetben, ha a TomoPy által biztosított numerikus Radon-transzformáció megoldását tekintjük referenciának, és a másik két alternatív módszer eredményeit ehhez viszonyítva elemezzük, akkor célszerű a különbségképek előállításából kiindulnunk.

Az 4.3. ábra világosan szemlélteti, hogy a szintén *rect* bázisfüggvényeket feltételező TomoPy és a saját implementációm különbségképe alacsony intenzitású. A 4.1. táblázat pedig számszerűen rámutat arra, hogy az eltérés (MSE) értéke alacsony ($2.595560e^{-06}$). Ezzel szemben a *circ* bázisfüggvényeket feltételezve jól látható a 4.3. ábra jobb alsó

sarkában, hogy a körív mentén számottevő differenciák mutatkoznak. A 4.1 táblázat alapján az eltérés értéke (7.741452), ami a *rect* bázisfüggvényeket feltételezőhöz viszonyítva jóval magasabb érték, ami jelentős eltérést jelez.

A továbbiakban az egyszerűség kedvéért azt a programváltozatot, amely a *circ* típusú bázisfüggvényeket feltételezi, CIRC-ként említjük, illetve a *rect* típusú bázisfüggvényeket feltételező programváltozatra pedig RECT-ként hivatkozunk.



4.3. ábra. A TomoPy könyvtárral előállított szinogram (felső sor), a RECT (második sor, bal) és CIRC (harmadik sor, bal) szinogramjai, valamint mellettük a TomoPy-hoz képest mért különbségképeik (második és harmadik sor, jobb) láthatóak.

A vizuális összehasonlítás mellett a különbségek számszerű elemzése is megtörtént, hogy pontosabb képet kapjunk az eltérésekről. Ennek érdekében a Mean Squared Error (MSE) mutató szolgál alapul, amely egy mérték a képpontok közötti intenzitáskülönbségek értékelésére. Az MSE kiszámítása az alábbi képlettel történik [3]:

$$MSE = \frac{1}{K} \sum_k (c_k - \hat{c}_k)^2 \quad (4.1)$$

A képlet azt fejezi ki, hogy minden egyes képpontra kiszámításra kerül az eredeti (c) és az összehasonlított kép ($\hat{c}$) megfelelő képpontjainak intenzitáskülönbsége, majd ennek a különbségnek a négyzetes átlagát vesszük.

A vizuális és az MSE alapú értékelés mellett a Mean Absolute Error (MAE) mutató is alkalmazásra került. Az utóbbi mérőszám az abszolút intenzitáskülönbségek átlagát adja meg, így jól tükrözi az egyes képpontok közti eltérések nagyságát anélkül, hogy a nagyobb hibákat négyzetre emelné. A MAE a következő képlettel számítható [3]:

$$MAE = \frac{1}{K} \sum_k |c_k - \hat{c}_k| \quad (4.2)$$

Minden képpontra kiszámoljuk az intenzitáskülönbség abszolút értékét, majd ezeknek az abszolút eltéréseknek az átlagát vesszük. Az MAE értéke közvetlenül mutatja a képpontok átlagos eltérését, és kevésbé erősíti fel a nagyobb hibákat, mint az MSE.

A negatív és pozitív szélsőértékek (minimum és maximum) alapján meghatározásra került a Maximum Absolute Error (MaxAE) értéke, amely az egész mátrixot tekintve a legnagyobb abszolút intenzitáskülönbséget tükrözi. Pontos képet ad arról, hogy a vizsgált képen mekkora a legnagyobb eltérés. Ez a következőképpen írható le [3]:

$$MaxAE = \max_{0 \leq k < K} |c_k - \hat{c}_k| \quad (4.3)$$

Az alábbi táblázat segítségével részletesen nyomon követhető a különböző módszerek eltérése.

	TomoPy - RECT	TomoPy - CIRC
Min	-0.016601	-23.929931
Max	0.011718	22.201995
MaxAE	0.016601	23.929931
MAE	0.000902	1.654987
MSE	$2.595560e^{-06}$	7.741452

4.1. táblázat. TomoPy és a RECT, illetve TomoPy és CIRC szinogram különbségeinek számszerűsített eltérései a 4.1. ábra látható kép alapján.

A RECT módszer esetében az eltérések értékei viszonylag alacsonyak, ami arra utal, hogy a TomoPy szinogram eredményéhez képest az eltérés elhanyagolható, így a két eljárás eredményei szinte teljes mértékben megegyeznek. Ezzel szemben a CIRC módszer esetén az eltérések nagyobbak.

Összességében a 4.1. táblázat egyértelműen rávilágít arra, hogy a hasonló elveken alapuló RECT szinte megegyező eredményt ad a TomoPy numerikus Radon-transzformáció megoldásához, míg a CIRC módszer a feltételezett bázisfüggvények különbségei miatt nagyobb eltérést mutat, mivel a folytonos körívet nem követi szorosan a képpontrács.

Ezen felül elkészült a 4.2. táblázat is, amely ötven darab, külön-külön 256×256 pixeles képek szinogramjaira vonatkozó átlagolt adatokat tartalmaz. Minden kép 101 különálló kört ábrázol. A vetítések száma megegyezett a képméret kétszeresével. A táblázat azt mutatja meg, hogy a szinogramok közötti különbségek MSE értéke hogyan alakul átlagosan az ötven kép esetében, valamint tartalmazza a minimum és maximum értékeket is.

	TomoPy - RECT	TomoPy - CIRC
MSE	$2.70564e^{-6}$	8.118256
Min	$1.79263e^{-6}$	6.820056
Max	$3.29241e^{-6}$	9.451903

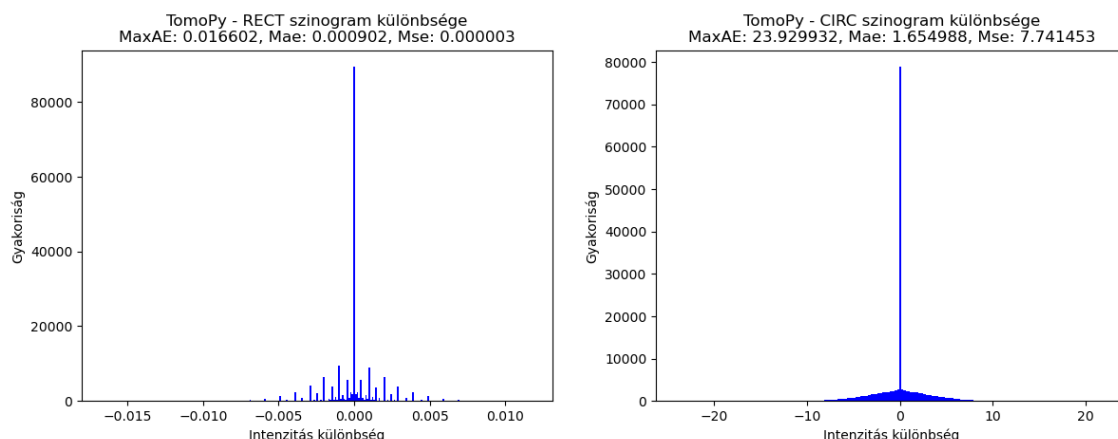
4.2. táblázat. *TomoPy és a RECT, illetve TomoPy és CIRC szinogram különbségeinek számszerűsített értékei, 50 kép eredményének átlaga alapján.*

Az ötven képre átlagolt eredmények azt mutatják, hogy egyik eljárás esetén sincs kiugró adat, az eltérések stabilan egy szűk tartományon belül maradnak. Ezek az eredmények azt igazolják, hogy mind a RECT, mind a CIRC módszer megbízhatóan, következetesen működik, az átlag körül mozogva.

A különbségek eloszlásának szemléltetésére hisztogramok is szolgálnak, amelyek vizuálisan ábrázolják, hogy az adott tartományba eső eltérések milyen arányban fordulnak elő.

A 4.4. ábra bal oldali hisztogramja (TomoPy – RECT szinogramok különbsége) szűk, szimmetrikus eloszlást mutat, ahol a legtöbb eltérés a nullához közel koncentrálódik, ami a két módszer közötti egyezést mutatja meg. A "fésűs" megjelenés a RECT hisztogramon

feltehetően a raszterizált számítási modellből adódik, a pixelrács pontos követése és a rögzített lépésköz olyan rendszeres kerekítési hibákat eredményez, amelyek periodikus csúcsokként jelennek meg. Ennek ellenére az eltérések nagysága elhanyagolható.

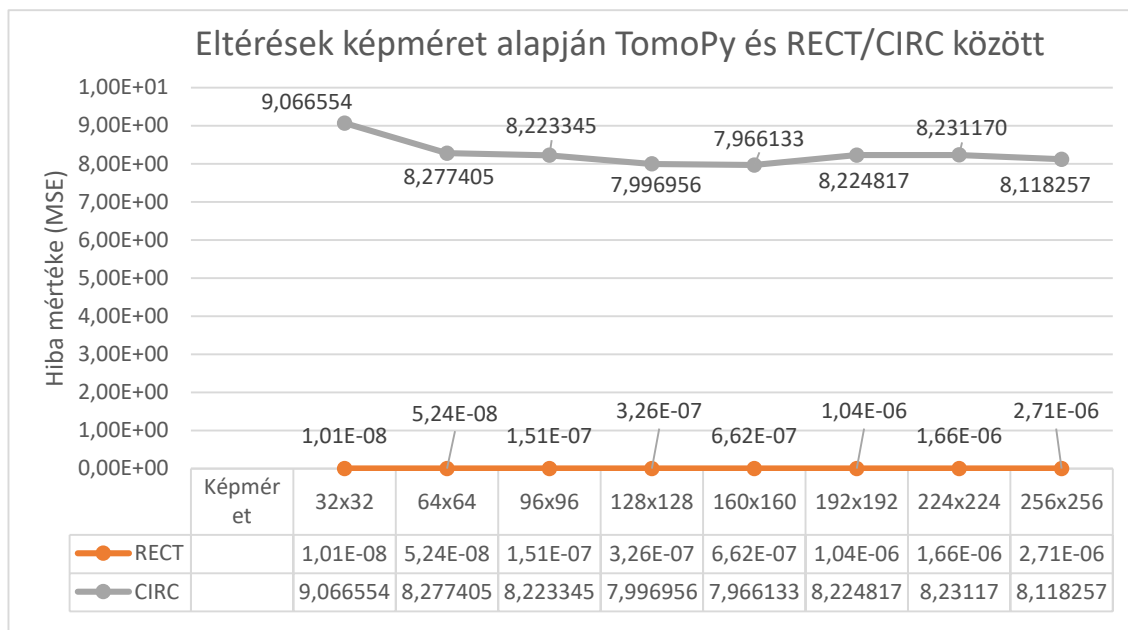


4.4. ábra. TomoPy és a RECT/CIRC módszerek közötti intenzitáseltérések ábrázolása hisztogramon.

Ezzel szemben a jobb oldali hisztogram (TomoPy – CIRC szinogramok különbsége) széles eloszlást mutat, nagyobb eltérésekkel. Ahogy az eltérés értéke közelít a nullához, úgy az eltérések gyakorisága fokozatosan emelkedik, majd magasan kiugrik a nullánál. Ez az alakzat jól szemlélteti, hogy bár előfordulnak eltérések, a leggyakoribb eset mégis az, amikor a két vetítés közel megegyező eredményt ad.

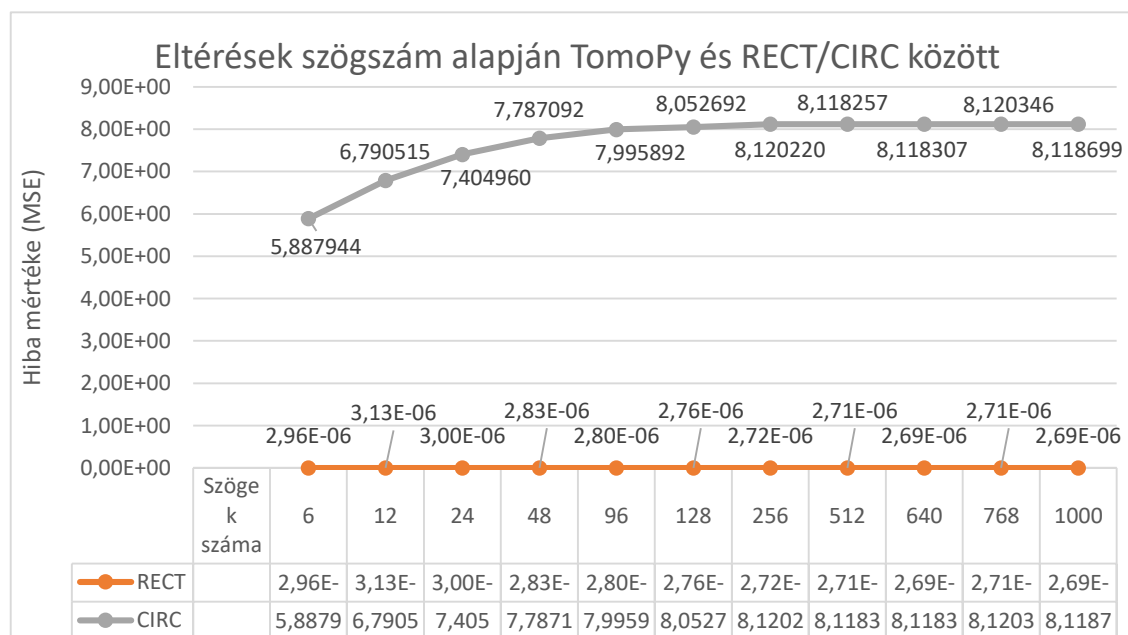
A szinogramok további vizsgálata a képméret és a vetítési szögek számának változtatásával történt. Ahogy a 4.2. táblázat esetében is, itt is ötven kép MSE értékeit átlagoljuk. Minden egyes konfiguráció esetében a TomoPy vetítő és a RECT, illetve a TomoPy vetítő és a CIRC eljárások közötti különbségeket hasonlítottuk össze. A módszer lehetővé teszi, hogy objektíven mérjük, miként változik a szinogramok közötti eltérés a bemenet felbontásának növelésével, illetve a vetítősugarak számának emelésével.

Elsőként a különböző képméretek esetén történt az összehasonlításra, majd a szögszámok szerint. A 4.1. diagram és a 4.2. diagramokon a TomoPy vetítő, a CIRC valamint a RECT eljárások különbségét hasonlítottuk össze a MSE metrika szerint. Vagyis a CIRC görbájén szereplő értékek úgy keletkeznek, hogy az MSE képletében (4.1) a c a TomoPy vetítő által előállított szinogramot, a $\hat{c}$ pedig CIRC szinogramját jelöli. Ugyanígy járunk el a RECT esetében is, ahol az MSE képletében a c a TomoPy vetítő által előállított szinogramot, a $\hat{c}$ pedig RECT szinogramját jelöli.



4.1. diagram. TomoPy vetítő – RECT és Tomopy vetítő - CIRC 50 szinogramjának átlagolt MSE értékei nyolc eltérő képméret alapján.

A táblázat alapján a TomoPy – RECT módszer MSE értékei alacsonyak maradtak és bár a felbontás növekedésével enyhe emelkedést mutatnak, az eltérések továbbra is kismértékűek. Ezzel szemben a CIRC eljárás kisebb felbontásnál magasabb MSE értékeket mutat, majd stabilizálódik.

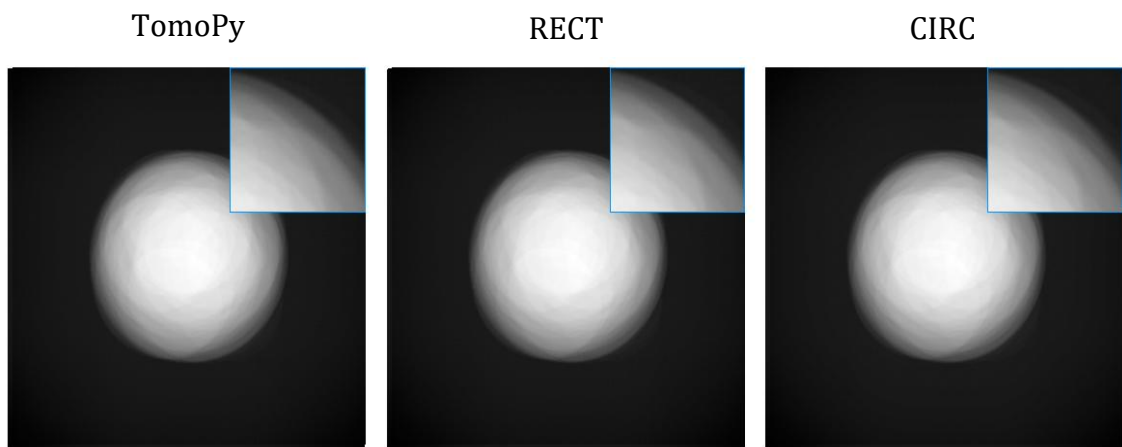


4.2. diagram. TomoPy vetítő – RECT és Tomopy vetítő - CIRC 50 szinogramjának átlagolt MSE értékei tizenegy eltérő szögszám alapján.

A szögszám növelésével a TomoPy vetítő – RECT MSE értékei elhanyagolható mértékben ingadoznak, de továbbra is szűk tartományban maradnak, míg a TomoPy vetítő – CIRC különbsége a kezdeti értékről gyorsan megnőtt, majd nagyjából egy érték köré összpontosult. Ez azt mutatja, hogy a RECT módszer szögfelbontásra kevésbé érzékeny, míg a CIRC eljárás esetén a szögszám növelése növeli az eltérés mértékét.

4.3 Kép rekonstruálása a szinogramból

A következő lépésben a három módszerrel előállított szinogramok mindegyikén külön-külön alkalmazásra kerül a *tomopy.recon* függvényhívás (A rekonstrukció a TomoPy csomag *gridrec* algoritmusával, *cosine* típusú szűrő alkalmazásával történt), amelynek célja az eredeti tesztkép minél pontosabb visszaállítása. A rekonstrukciók elemzése lehetőséget nyújt arra, hogy részletesen összehasonlítsuk az egyes implementációk közötti különbségeket. Az így létrejött rekonstruált képek alapján értékelhető, hogy az egyes módszerek milyen mértékben képesek megőrizni és visszaadni az eredeti kép információtartalmát.



4.5. ábra. *TomoPy gridrec rekonstrukciós algoritmus által rekonstruált képek. Balról jobbra haladva a TomoPy vetítő, RECT, illetve CIRC szinogramjaiból, a 4.1 tesztkép alapján.*

Mivel az eredmények közötti eltérések olyan kismértékűek, hogy a képi megjelenítés során szabad szemmel nem láthatóak, így az összehasonlítás megkönnyítése érdekében itt is célszerű az eltéréseket számszerűsíteni. Ennek megfelelően lentebb a rekonstruált és az eredeti képek közötti eltérések táblázatos formában kerülnek bemutatásra.

Ezek alapján az alábbi táblázat a 4.5. ábra látható három eltérő módszer hibamértékeit mutatja be. A tesztek során az MSE, MAE, MaxAE valamint a Min és Max kerültek kiértékelésre.

	TomoPy	RECT	CIRC
Min	-8.708623	-8.708606	-8.749890
Max	7.028347	7.028411	7.394077
MaxAE	8.708623	8.708606	8.749890
MAE	4.050966	4.050965	4.052030
MSE	17.936521	17.936509	17.944801

4.3. táblázat. *TomoPy gridrec rekonstrukciós algoritmusát felhasználva a TomoPy vetítő, RECT, illetve a CIRC szinogramjaiból rekonstruált képek hibaértékei. Az eredeti kép a 4.1. ábra látható.*

Összességében a táblázat azt mutatja, hogy a TomoPy rekonstrukciós algoritmus és a RECT módszer eredményei gyakorlatilag megegyeznek, az eltérések minimálisak, mind a Min, Max, MAE és MSE értékek szinte azonosak.

Ezzel szemben a CIRC módszer valamivel nagyobb eltéréseket mutat, a Min, Max értékek is kicsit szélesebb tartományban helyezkednek el, ami együtt jár a MaxAE érték romlásával. A MAE és MSE értékek pedig enyhén magasabbak.

A rekonstrukciós eljárásra is készült átlagoló táblázat, amely a szinogram összesítő 4.2. táblázat szinogram adataiból épült fel. Vagyis ötven darab szinogramból előállított rekonstruált kép átlagolt adatait tartalmazza. Minden kép 101 különálló kört ábrázol.

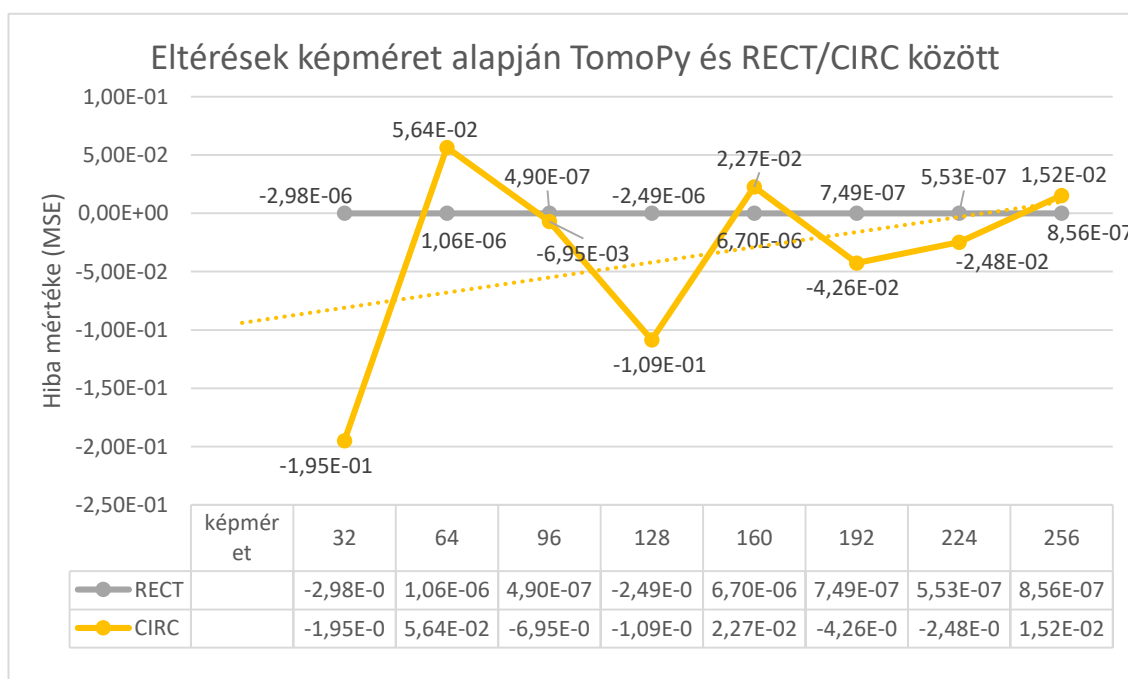
	TomoPy	RECT	CIRC
MSE	19.560133	19.560132	19.544943
Min	14.202173	14.202189	14.208338
Max	24.341695	24.341720	24.420073

4.4. táblázat. *A TomoPy gridrec rekonstrukciós algoritmusát felhasználva, a TomoPy vetítő, RECT, illetve a CIRC szinogramjaiból rekonstruált képek hibaértékei. Ötven 256x256 felbontású kép MSE eredményének átlaga alapján, ahol a vetítősugarak száma megegyezik a kép szélességének kétszeresével.*

A fenti táblázat azt mutatja meg, hogy a rekonstruált képek közötti különbségek, pontosabban MSE értéke hogyan alakul átlagosan az ötven eltérő kép esetén. Ezen felül a táblázat tartalmazza a tesztek során mért legalacsonyabb, illetve legmagasabb hibaértékeket is, ami felfogható két szélsőértéknek.

A 4.4. táblázat adatai alapján mindhárom eljárás nagyon hasonló eredményeket mutat, viszont a CIRC ebben az átlagolt esetben, csekély mértékben pontosabb eredményeket adott a rekonstrukciók során. Az MSE értékek alapján elhanyagolható eltérés tapasztalható a TomoPy gridrec rekonstrukciós algoritmusával készült képeknél, ahol a TomoPy vetítő, valamint a RECT szinogramjaiból indultunk ki. A minimum és maximum értékek is szinte azonosak.

A CIRC esetében a minimum és maximum értékek nagyobb tartományt vesznek fel, viszont az átlagolt MSE kisebb pár századdal. Ezek alapján megállapítható, hogy a különböző szinogramokból készült rekonstruált képek közel azonos minőségűek, és egyik módszer sem mutat igazán jelentős előnyt vagy hátrányt a másikhoz képest.



4.3. diagram. A TomoPy gridrec algoritmusával végzett rekonstrukciók esetén meghatározásra került az MSE értékek közti különbség a TomoPy vetítő és a RECT szinogramjai alapján készült képekre. Ugyanez a CIRC esetében is kiszámításra került. Az eredmények 50 kép MSE értékének átlagolásával adódtak. Az összehasonlítás nyolc különböző képméret szerint történt.

A 4.3. diagram a MSE értékeinek különbségeit tükrözi. A különböző felbontások mellett az eltérés vizsgálatára azért volt szükség, mert a 256×256-os képméret esetén

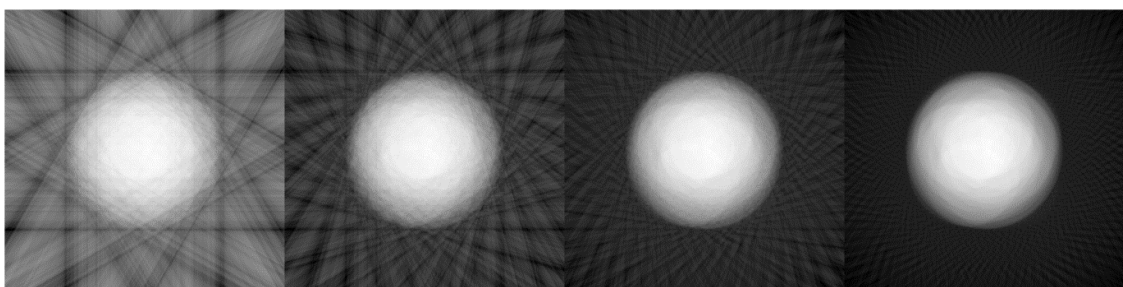
nincs jelentős különbség. A TomoPy gridrec algoritmus felhasználásával háromféle szinogramból (TomoPy vetítő, RECT, CIRC) készültek rekonstrukciók. Ezután mindegyik esetben kiszámítottam a rekonstruált és az eredeti tesztkép közti MSE értéket. Mivel az egyes módszerek eredményei nagyon hasonlóknak bizonyultak, az MSE különbségét ábrázoltam.

Az adatok ötven különböző képen végzett mérések átlagolásával készültek, különböző képméretek esetén.

A RECT módszer esetén az eltérések szórása szűkebb tartományban mozog, és az MSE értékek kisebb mértékben térnek el. Ezzel szemben a CIRC módszernél különösen a kisebb képméretek esetén figyelhetők meg nagyobb és változékonyabb eltérések. Ez arra enged következtetni, hogy a CIRC eljárás kisebb képméreteknél hajlamosabb nagyobb rekonstrukciós hibákat eredményezni (véltetően a raszterizáció miatt).

4.4 A vetületi szögek számának jelentősége rekonstrukció esetén

A tesztek során egyúttal arra is kiterjedt a vizsgálat, hogy a szögek számának csökkentése milyen hatással van a rekonstruált képre. Ahogy a sugarak forgatási szögeinek száma csökken, egyre kevesebb részlet válik láthatóvá a rekonstruált képen, és az egyes részletek elmosódása fokozódik. Ezt a jelenséget illusztrálja a 4.6. ábra, ahol világosan látszik, hogy a vetítési szögek számának csökkenése jelentős mértékben befolyásolja a kép minőségét.

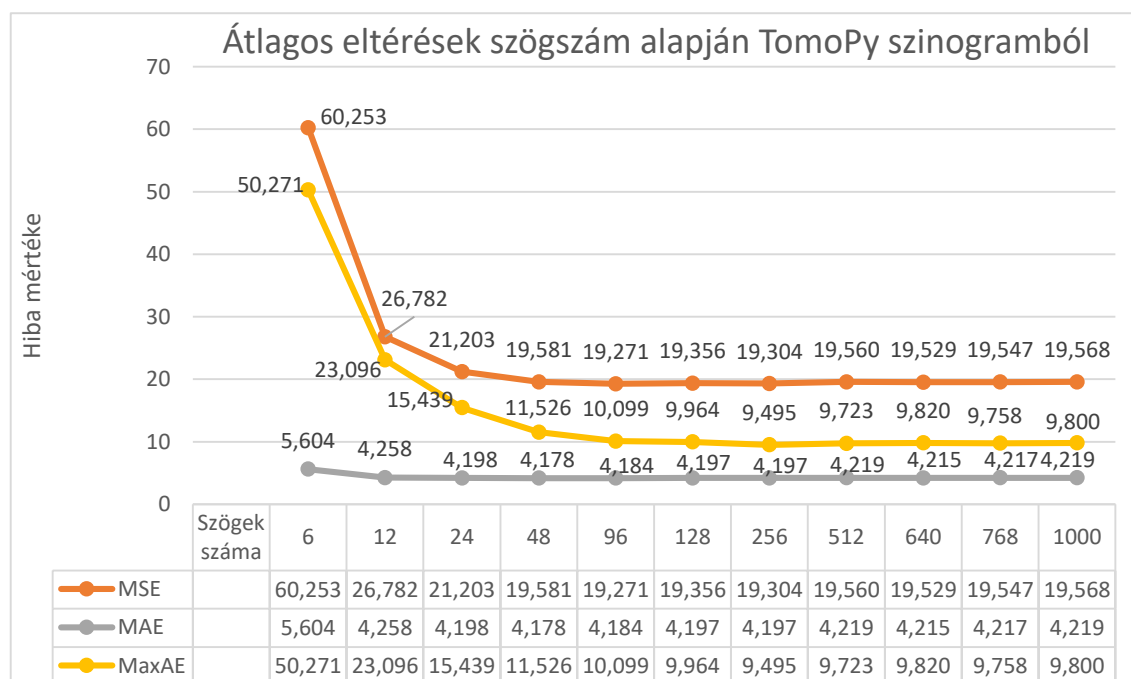


4.6. ábra. *TomoPy rekonstrukció eredményei 6-12-24-48 darab forgatási szög esetén a 4.1. ábra látható tesztkép alapján.*

A képeken jól megfigyelhető, hogy minél kevesebb forgatási szög áll rendelkezésre a rekonstrukcióhoz, annál inkább romlik az eredmény minősége, és az eredeti kép egyre elmosódottabbá, felismerhetetlenebbé válik. Már 48 forgatási szög esetén is érzékelhető némi részletvesztés, de a főbb formák még jól kivehetők. 24 szög alkalmazásánál a

részletek tovább mosódnak, 12 szögnél már jelentős minőségromlás tapasztalható, míg 6 szög esetében a rekonstrukció erősen elmosódottá válik. Ez az eredmény alátámasztja azt az általános elvet, hogy a vetítések számának növelése javítja a rekonstrukció minőségét egy bizonyos pontig, míg a túl kevés szög jelentős információvesztést eredményez.

A TomoPy könyvtár segítségével a *tomopy.project* függvénnyel előállított szinogramot a hozzá tartozó *tomopy.recon* függvénnyel rekonstruáltam (algorithm='gridrec', filter_name='cosine'), így egy az eredeti képet megközelítő rekonstrukció áll elő. Mivel mindkét függvény ugyanabból a csomagból származik, feltételezhető, hogy a rekonstrukció pontos eredményt ad, ezért az így kapott eredményeket az eredeti képekkel összevetve értékelem a hibákat.

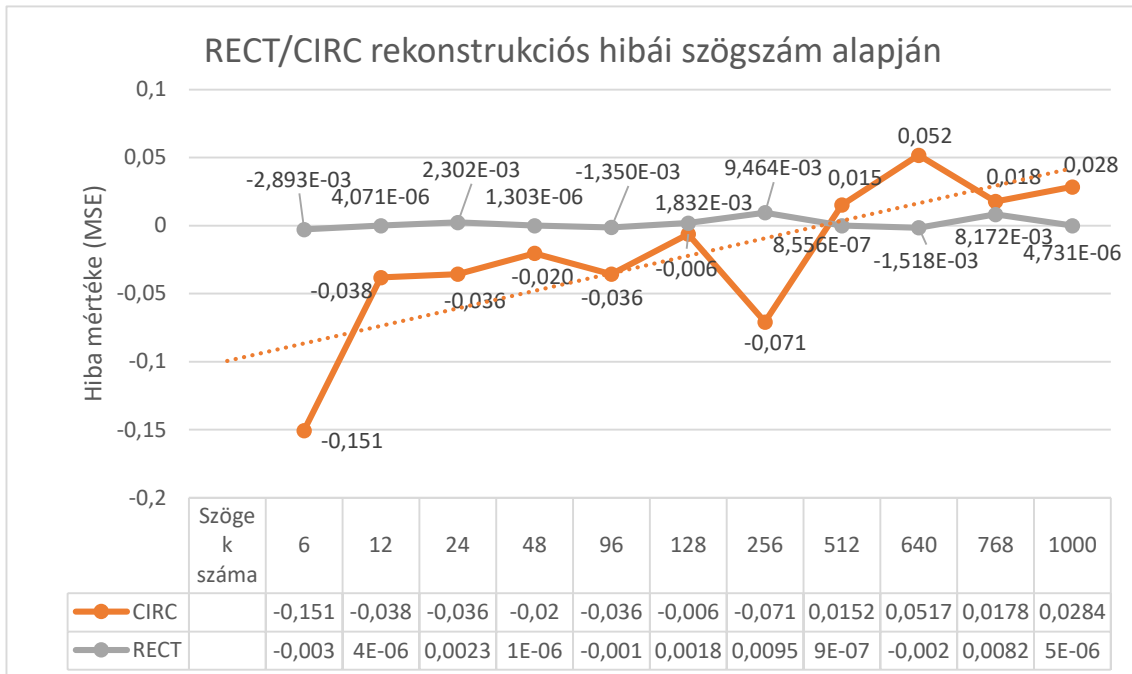


4.4. diagram. *TomoPy rekonstrukciós algoritmusa által készült ötven rekonstrukció átlag hibaértékei. Tizenegy különböző szögszám alapján.*

A diagram egyértelműen szemlélteti, hogy a rekonstrukció minősége szoros összefüggést mutat a vetítési szögek számával. A görbén látható, hogy a képminőség fokozatosan javul a forgatási szögek számának növelésével, majd egy bizonyos szint (körülbelül a kép méretével azonos számú szög) elérése után a további javulás mértéke lelassul/ megáll. Ez azt jelzi, hogy a kellően nagy szögszám vetítés után már nem érhető el további javulás, míg korábban, alacsony szögszámok esetén minden további vetítési szög érdemi információt ad hozzá a rekonstrukcióhoz. A tapasztalatok alapján tehát a rekonstruált kép

optimális minősége akkor biztosítható, ha a vetítési szögek száma legalább a kép szélességével megegyező értéket eléri, utána már nem tapasztalható számottevő javulás.

Ezt követően a 4.5. diagramon a különböző típusú szinogramokból rekonstruált képeket MSE értékeit eltérő szögszámok esetén ábrázoltam.



4.5. diagram. A TomoPy gridrec algoritmusával végzett rekonstrukciók esetén meghatározásra került az MSE értékek közti különbség a TomoPy vetítő és a RECT szinogramjai alapján készült képek, valamint az eredeti tesztkép viszonylatában. Majd ugyanez a CIRC esetében is kiszámításra került. Az MSE értékkülönbségek 50 mérésen alapuló átlag alapján készültek, tizenegy különböző szögszámra.

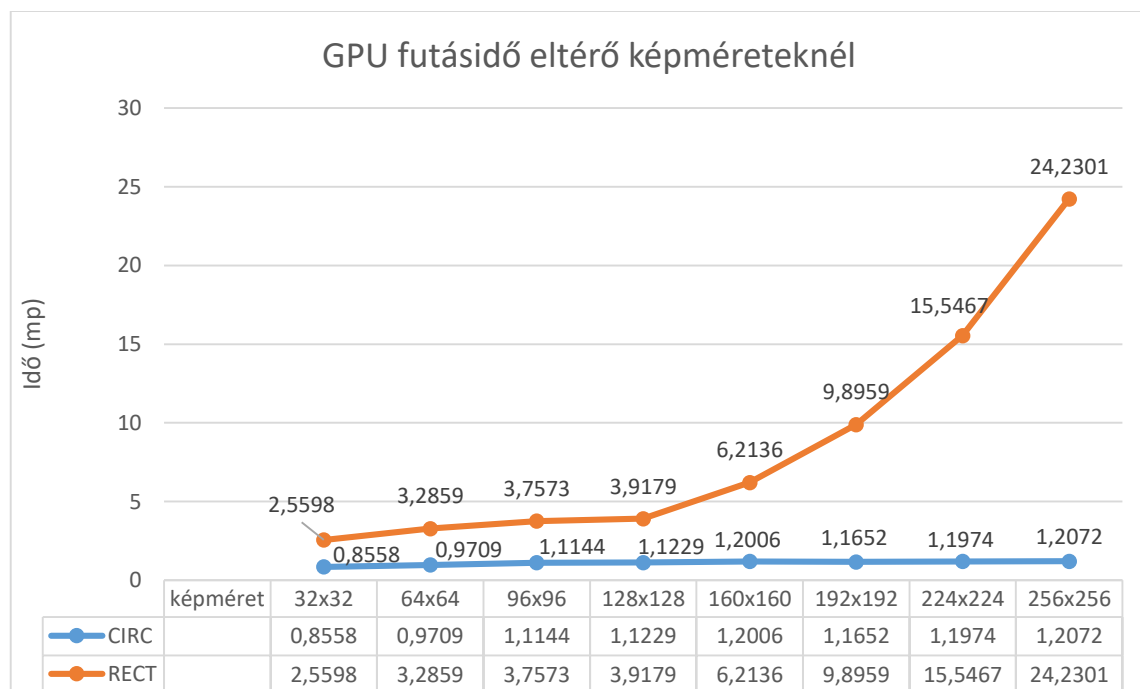
A fenti diagram azt mutatja, hogy a három féle szinogramból a TomoPy gridrec rekonstrukciós algoritmusának eredményei, hogyan alakulnak eltérő szögszámok esetén. A folyamat is itt olyan módon került számításra, mint a 4.3. diagram esetén. A mérés itt is 50 képen történt, majd ezek átlagát vettem.

A CIRC hibái jóval nagyobb szórást mutatnak, különösen 6 szögszám esetén, ahol jelentősebb eltérések figyelhetők meg a TomoPy vetítő szinogramjának rekonstrukciójához képest. A RECT hibái ezzel szemben kisebbek, és jellemzően közelebb vannak a nullához, különösen alacsonyabb szögszám esetén. Ez arra utal, hogy főleg kisebb szögszámokra a RECT közel megegyező eredményeket ad, míg a CIRC hibái csak egy bizonyos szögszám után kezdenek kiegyenlítődni.

4.5 Futásidő elemzés

A számítási komplexitás és a program futásidejének vizsgálata kulcsfontosságú tényező annak mélyreható megértéséhez, hogy a különböző Radon-transzformációs módszerek milyen hatékonysággal és skálázhatósággal működnek. Így lehetővé válik a módszerek összehasonlítása mind nagy felbontású, mind valós idejű alkalmazásokban. Végző soron a komplexitás és futásidő vizsgálat átfogó képet nyújt a módszerek gyakorlati értékéről, a párhuzamosíthatóság lehetőségeiről és a várható skálázódási viselkedésről.

A tesztek i5-9300H CPU-n és NVIDIA GeForce GTX 1650 Mobile GPU-n történtek, ami egy közepes teljesítményű grafikus kártya, viszont alkalmas ilyen jellegű számítások futtatására.



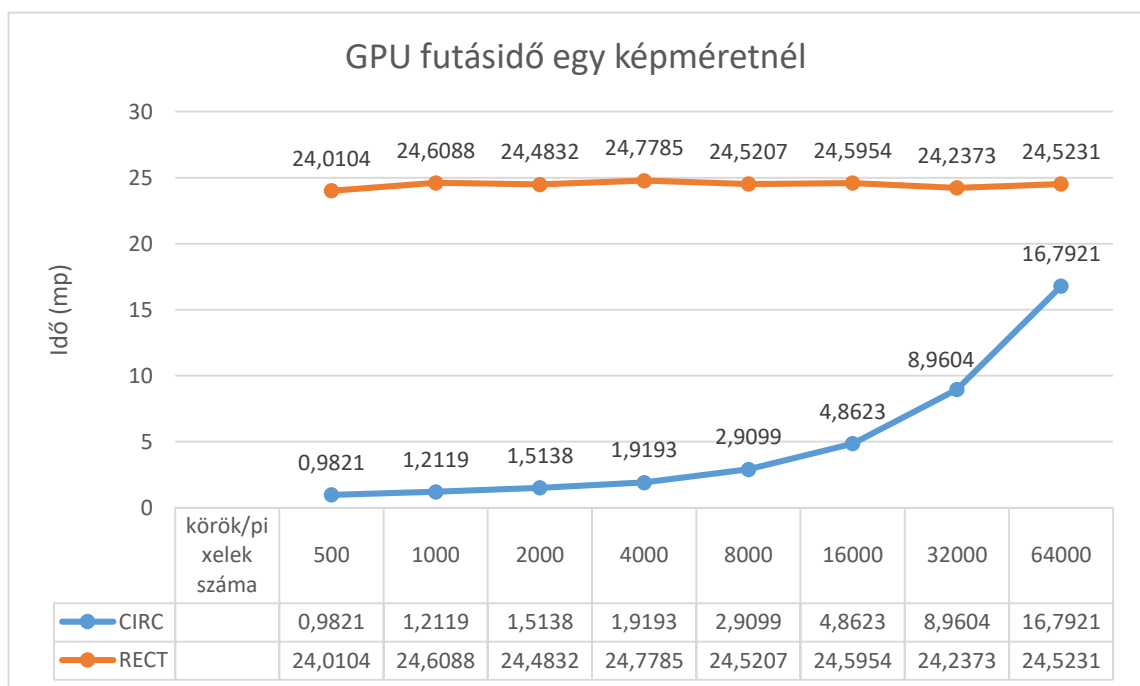
4.6. diagram. A CIRC és RECT GPU futásideje eltérő képméretek esetén, miközben a körök/pixelek száma állandó.

Az elemzés során a kör/pixelszám állandó 900-as értéket vesz fel, ami mellett a kép felbontásának fokozatos növelésével (32×32-től 256×256-ig) kerülnek tesztelésre a módszerek.

A mért futási idők mindkét algoritmus esetében egyértelműen megmutatják, hogy a CIRC módszer viszonylag független a felbontás növelésével szemben, az időértékek 0,86 mp-től 1,21 mp-ig terjednek. Ez arra utal, hogy a körönkénti számítási költség

domináns, a memóriáhozáférés és párhuzamosítás szemszögéből. Ezzel ellentétben a RECT algoritmus futásideje meredeken emelkedik a pixelek számával, hiszen a pixelenkénti műveletek száma arányos a képpontok összmennyiségével, így a 2,56 mp-től induló érték 24,23 mp-ig kúszik fel. A látottak alapján nagy felbontású bemeneteknél a CIRC algoritmus skálázódása kiszámíthatóbb és erőforrás szempontjából hatékonyabb.

A CIRC módszer viselkedésének részletesebb feltérképezése érdekében a kép felbontásának változtatásától elvonatkoztatva kizárólag a körök/pixelek számát módosítjuk. Ezzel a kísérleti beállítással lehetővé válik, hogy a futásidő és a bemeneti körszám/pixelszám közötti lineáris összefüggés is jól láthatóvá váljon.



4.7. diagram. A CIRC és RECT GPU-n lévő futásideje a eltérő körök/pixelek száma alapján, állandó képméret mellett.

Ebben az esetben szintén GPU-n történtek a számítások 256×256 tesztképekre, ahol világosan látszik a két módszer eltérő viselkedése. A CIRC futási ideje a kör/pixel növelésével közel lineárisan emelkedik (0,98 mp-ről 500 körtől , egészen 16,79 mp-ig 64 000 körig), ezzel szemben a RECT futásideje a pixelek változtatásától függetlenül nagyjából állandó, 24–25 másodperc körül ingadozik. Ez arra utal, hogy a RECT módszer számítási ideje elsősorban a 256×256-os kép felbontás határozza meg, míg a CIRC algoritmus komplexitása egyértelműen $O(k)$, azaz arányos a bemeneti körök számával. Ennek megfelelően alacsony vagy közepes kör/pixelszámok esetén a CIRC egyértelműen

hatékonyabbnak bizonyult. Ugyanakkor nagy adatmennyiség, azaz magas kör/pixelszám esetén a RECT futási ideje kedvezőbbé válhat, így ilyen esetekben az alkalmazása előnyösebb lehet.

Összefoglalva a RECT algoritmus futásideje a képpontok számával skálázódik, hiszen minden pixelre külön műveletet kell végezni, így a nagyobb felbontás drasztikusan növeli a végrehajtási időt, viszont a kép felbontása nagyban befolyásolja a futásidejét. Ezzel szemben a circ módszer lineáris viselkedést mutat a körök számával $O(k)$. A képméret változása csak minimális hatással van a végrehajtási időre, mivel a számítási terhelés döntő részét a körök feldolgozása okozza.

Összefoglalás

A bemutatott mérési eredmények alapján megállapítható, hogy bár a Radon-transzformáció elméleti alapjai egységesek, a különböző bázisfüggvények alkalmazása számottevő hatással van a gyakorlati eredményekre. Első pillantásra a két módszer a CIRC és a RECT hasonló vetületi adatokat eredményez, részletesebb vizsgálat során jól érzékelhető eltérések figyelhetők meg. Ezek nemcsak a számított szinogramok szerkezetében, hanem a rekonstruált képek minőségében és az algoritmusok számítási teljesítményében is megmutatkoznak.

A CIRC alapú eljárás során előállított szinogramok jellemzően simább, fokozatosabb átmeneteket mutatnak, amennyiben a kép eleve köralakú objektumokból áll vagy jórészt ilyen objektumok alkotják, ami a bázisfüggvény geometriai tulajdonságaiból adódik. Ezzel szemben a RECT alapú, módszer élesebb, szögletes mintázatot eredményez, különösen az objektumhatárok közelében. Ezek az eltérések azonban nem vezetnek torzuláshoz a vetületi képek általános mintázatában, ugyanakkor finom részletek vizsgálatakor már számottevő különbséget mutatnak.

A vetítési szögek számának növelése mindkét módszernél javította a rekonstrukció minőségét, azonban ez a javulás egy adott ponton túl már nem hozott jelentős változást. A gyakorlatban ez azt jelenti, hogy létezik egy optimális szögtartomány, amely felett a további növelés nem jár minőségjavulással.

A futásidő szempontjából a CIRC módszer előnyt mutatott, viszonylag kevés kör esetén. A mérések azt bizonyították, hogy gyorsabban hajtotta végre a vetületi számításokat, és a képméret növekedésével a futásidő csak enyhén emelkedett. Ezzel szemben a RECT esetében a képméret növekedése jelentősebb futásidő növekedést eredményezett. Viszont amikor a felbontást változatlanul hagytuk, és kizárólag a körök/pixelek számát növeltük, a tendencia megfordul.

Összességében tehát megállapítható, hogy a CIRC megközelítés bizonyos esetekben hatékonyabb lehet számítási szempontból. A kép geometriájától függően vetületi és visszafejtett képek természetesebb, simább megjelenését is elősegíti. Míg a RECT módszer jól alkalmazható diszkrét, raszterizált adatok feldolgozására.

Irodalomjegyzék

- [1] Balázs Péter. Képrekonstrukció. 2011.
URL: <https://eta.bibl.u-szeged.hu/307/>
Utolsó megtekintés dátuma: 2025.05.06.
- [2] Beylkin Gregory. The inversion problem and applications in tomography. 1987.
URL: <https://amath.colorado.edu/~beylkin/papers/BEYLKI-1987.pdf>
Utolsó megtekintés dátuma: 2025.05.06.
- [3] Botchkarev Alexei. A new typology design of performance metrics to measure errors in machine learning regression algorithms. 2019.
URL: <https://www.ijikm.org/Volume14/IJIKMv14p045-076Botchkarev5064.pdf>
Utolsó megtekintés dátuma: 2025.05.06.
- [4] Colin Flanagan. The Bresenham Line-Drawing Algorithm.
URL: <https://www.cs.helsinki.fi/group/goa/mallinnus/lines/bresenh.html>
Utolsó megtekintés dátuma: 2025.05.06.
- [5] CUDA Community. CUDA with Numba Documentation.
URL: <https://numba.pydata.org/numba-doc/dev/cuda/index.html>
Utolsó megtekintés dátuma: 2025.05.06.
- [6] Matplotlib Community. Matplotlib Documentation.
URL: <https://matplotlib.org/stable/index.html>
Utolsó megtekintés dátuma: 2025.05.06.
- [7] Dr. Nagy Gábor. Koordinátageometria jegyzetvázlat. 2002.
URL: <https://www.math.u-szeged.hu/~nagyg/hu/assets/PDF/koordgeo.pdf>
Utolsó megtekintés dátuma: 2025.05.06.
- [8] NERSC. TomoPy Performance Case Study.
URL: <https://docs.nersc.gov/performance/case-studies/tomopy>
Utolsó megtekintés dátuma: 2025.05.06.
- [9] NumPy. NumPy Documentation.
URL: <https://numpy.org>
Utolsó megtekintés dátuma: 2025.05.06.
- [10] Panja Ananya. Two Dimensional Signals, 2009.
URL: https://www.projectrhea.org/rhea/index.php/2D_Signals-Rect%28x,y%29
Utolsó megtekintés dátuma: 2025.05.06.
- [11] TomoPy. TomoPy Documentation.
URL: <https://tomopy.readthedocs.io/en/latest/index.html>
Utolsó megtekintés dátuma: 2025.05.06.
- [12] Varga László Gábor. CUDA programozás Példatár.
URL: https://www.inf.u-szeged.hu/~vargalg/oktatas/gpgpu_19-20_1/GPGPU_peldatar/index.html
Utolsó megtekintés dátuma: 2025.05.06.

- [13] Varga László Gábor. Information Content of Projections and Reconstruction of Objects in Discrete Tomography. 2013.
URL: https://doktori.bibl.u-szeged.hu/id/eprint/1837/1/VL_thesis_main.pdf
Utolsó megtekintés dátuma: 2025.05.06.
- [14] Wettl Ferenc. Lineáris Algebra. 2011.
URL: <https://www.math.u-szeged.hu/~mmaroti/okt/2015o/Wettl%20-%20Linearis%20algebra.pdf>
Utolsó megtekintés dátuma: 2025.05.06.
- [15] Wolfram Research. Rectangle Function – MathWorld
URL: <https://mathworld.wolfram.com/RectangleFunction.html>
Utolsó megtekintés dátuma: 2025.05.06.

Nyilatkozat

Alulírott Pető József programtervező informatikus BSc szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Képfeldolgozás és Számítógépes Grafika Tanszékén készítettem, programtervező informatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel. Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

Szeged, 2025. május 10.

Pető József
hallgató

Köszönetnyilvánítás

Ezúton szeretném megköszönni Domokos Zoltánnak a szakmai támogatását és értékes tanácsait, amelyek nagyban hozzájárultak a szakdolgozatom elkészítéséhez. Ezenfelül hálás vagyok a családomnak is, akik végig mellettem álltak, biztattak és mindenben segítettek a dolgozat megírásának időszakában.

Mellékletek

A szakdolgozat tárolására használt GitHub Repository elérhetőségei:



9.1. ábra. *GitHub Repository QR Code*

<https://github.com/IPetoI/Szakdolgozat/tree/main>